

AdvantageKit

Getting Started

入门指南

English / 中文 · 22 篇文章

按照本地 AdvantageKit 文档导航顺序编排

包含 Welcome、What's New in 2026? 和 Getting Started 全部文章。

GIF 以静态帧呈现；视频保留在线链接。

Contents / 目录

Welcome / 欢迎	3
What's New in 2026? / 2026 年有哪些新功能?	4
What is AdvantageKit / 什么是 AdvantageKit	10
What is AdvantageKit? / 什么是 AdvantageKit?	10
Championship Conference / Championship 会议	12
The Basics: Output Logging / 基础: 输出日志记录	13
The Basics: Bug Fixes / 基础: 修复缺陷	17
The Basics: Rapid Iteration / 基础: 快速迭代	20
Installation / 安装	22
Installation / 安装	22
Existing Projects / 现有项目	24
Version Control / 版本控制	26
Welcome to AdvantageKit / 欢迎使用 AdvantageKit	29
Template Projects / 模板项目	31
2026 KitBot Template / 2026 KitBot 模板	31
Differential Drive Template / 差速底盘模板	32
Spark Swerve Template / Spark Swerve 模板	39
TalonFX(S) Swerve Template / TalonFX(S) Swerve 模板	50
Vision Template / 视觉模板	64
Skeleton Template / 骨架模板	68
Replay / 回放	69
◀ How To: Traditional Replay / ▶ 操作指南: 传统回放	69
How To: Replay Watch / 操作指南: Replay Watch	71
Common Issues / 常见问题	73
Non-Deterministic Data Sources / 非确定性数据源	73
Multithreading / 多线程	74
Uninitialized Inputs / 未初始化的输入	75

Welcome / 欢迎



AdvantageKit / AdvantageKit

AdvantageKit is a logging, telemetry, and replay framework developed by [Team 6328](#). AdvantageKit enables **deterministic log replay** in Java, allowing the full state of WPILib robot code to be efficiently replayed in simulation based on a log file ([What is AdvantageKit?](#)). AdvantageKit is **free and open-source** with no restrictions on vendor compatibility. See also:

AdvantageKit 是由 [Team 6328](#) 开发的日志记录、遥测和回放框架。AdvantageKit 在 Java 中实现了确定性日志回放，可以基于日志文件在仿真中高效地回放 WPILib 机器人代码的完整状态（[什么是 AdvantageKit?](#)）。AdvantageKit **免费且开源**，对厂商兼容性没有限制。另请参阅：

- **AdvantageScope**, Team 6328's robot telemetry application which **does not require AdvantageKit to use**.

AdvantageScope: Team 6328 的机器人遥测应用，使用它不需要 **AdvantageKit**。

- **WPILib Data Logging**, a simpler logging system included in WPILib (does not support log replay in simulation, but covers the needs of most teams).

WPILib 数据日志: WPILib 内置的更简单的日志系统（不支持在仿真中回放日志，但能满足大多数队伍的需求）。

Feedback, feature requests, and bug reports are welcome on the [issues page](#). See the [contribution page](#) for more information on contributing to AdvantageKit. For non-public inquiries, please send a message to software@team6328.org.

欢迎在 [issues 页面](#) 提供反馈、提出功能请求和报告缺陷。有关为 AdvantageKit 作贡献的更多信息，请参阅[贡献页面](#)。对于不便公开的咨询，请发送邮件至 software@team6328.org。

What's New in 2026? / 2026 年有哪些新功能?



AdvantageKit 2026 banner (dark theme) / AdvantageKit 2026 横幅（深色主题）

The 2026 version of AdvantageKit is now available! Check the [installation docs](#) and [full changelog](#) for details. In addition to adding support for WPILib 2026, this release includes several major new features and documentation improvements. The most notable features in this release are documented below.

AdvantageKit 的 2026 版现已发布！详情请参阅[安装文档](#)和[完整更新日志](#)。除增加对 WPILib 2026 的支持外，此版本还包含几项重要的新功能和文档改进。下面介绍本次发布中最值得关注的功能。

We value your feedback! Feedback, feature requests, and bug reports are welcome on the [issues page](#).

我们重视你的反馈！欢迎在 [issues 页面](#) 提供反馈、提出功能请求和报告缺陷。

Unit Logging / 单位记录

All logging interfaces now support specifying unit metadata compatible with [unit-aware graphing](#) in AdvantageScope. Several examples of unit metadata logging are shown in the code block below. For more details, check the documentation [here](#).

所有日志记录接口现在都支持指定单位元数据，并与 AdvantageScope 中的[支持单位的图表](#)兼容。下面的代码块展示了几个记录单位元数据的示例。更多详情请参阅[此处](#)的文档。

java

```
Logger.recordOutput("MyAngle", 3.141, "radians");
Logger.recordOutput("MyDistance", Meters.of(63.28));
Logger.recordOutput("MyCurrent", 42.0, Amps);

@AutoLog
public class Inputs {
    public Current current = Amps.of(63.28);
}

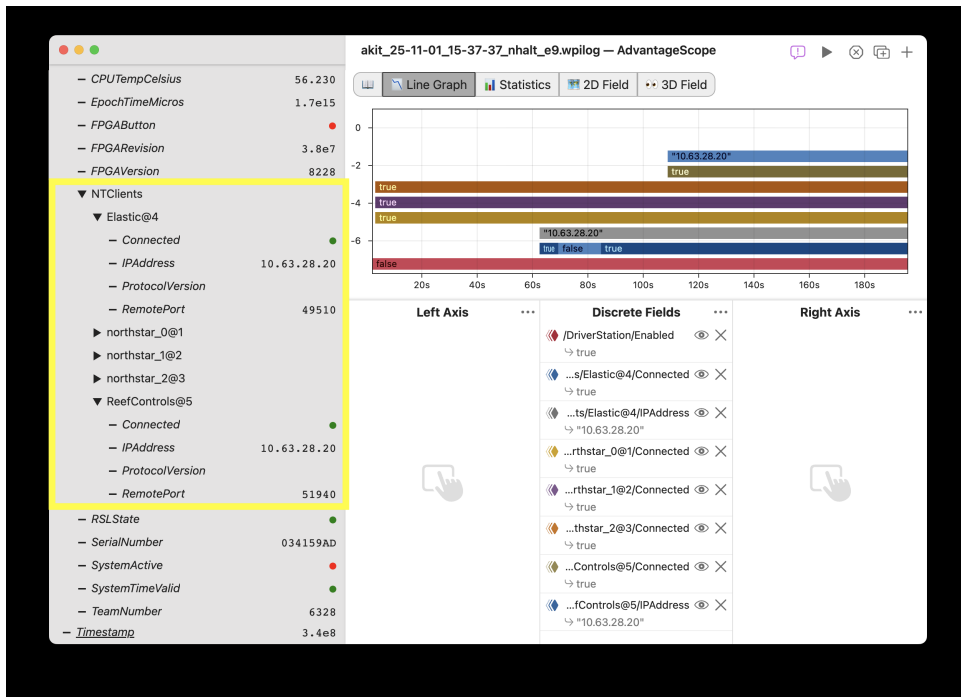
public class Outputs {
    @AutoLogOutput(unit = "inches")
    private double setpoint = 44600.0;

    @AutoLogOutput
    private Voltage volts = Volts.of(12.6);
}
```

NetworkTables Client Logging / NetworkTables 客户端记录

The SystemStats table now tracks which clients are connected to the NetworkTables server, including dashboards and vision coprocessors. This makes it easy to check when devices are online with no additional logging in user code. For each client, AdvantageKit automatically logs the connection status, IP address, remote port, and protocol version.

SystemStats 表现在会跟踪哪些客户端连接到了 NetworkTables 服务器，包括 dashboard 和视觉协处理器。这样无需在用户代码中增加日志记录，就能方便地检查设备何时在线。对于每个客户端，AdvantageKit 会自动记录连接状态、IP 地址、远程端口和协议版本。



Example of NetworkTables client logging / NetworkTables 客户端记录示例

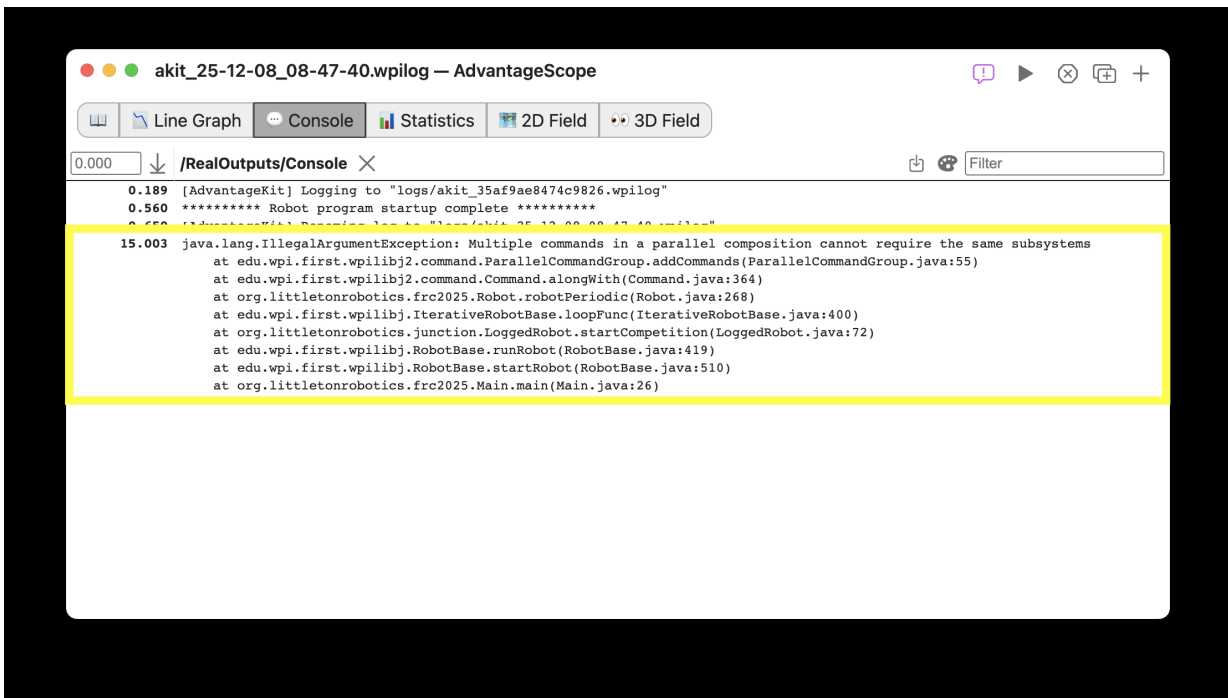
Improved Console Logging / 改进的控制台日志记录

AdvantageKit's [automatic console logging](#) has been improved to capture exceptions thrown during robot code execution. This makes it easier to debug code crashes without needing to reference the Driver Station log file.

AdvantageKit 的[自动控制台日志记录](#)已得到改进，可以捕获机器人代码执行期间抛出的异常。这样更容易调试代码崩溃问题，无需查看 Driver Station 日志文件。

In addition, console output is now displayed during [Replay Watch](#) to make it easier to debug robot programs when rapidly iterating on code.

此外，[Replay Watch](#) 期间现在也会显示控制台输出，便于在快速迭代代码时调试机器人程序。

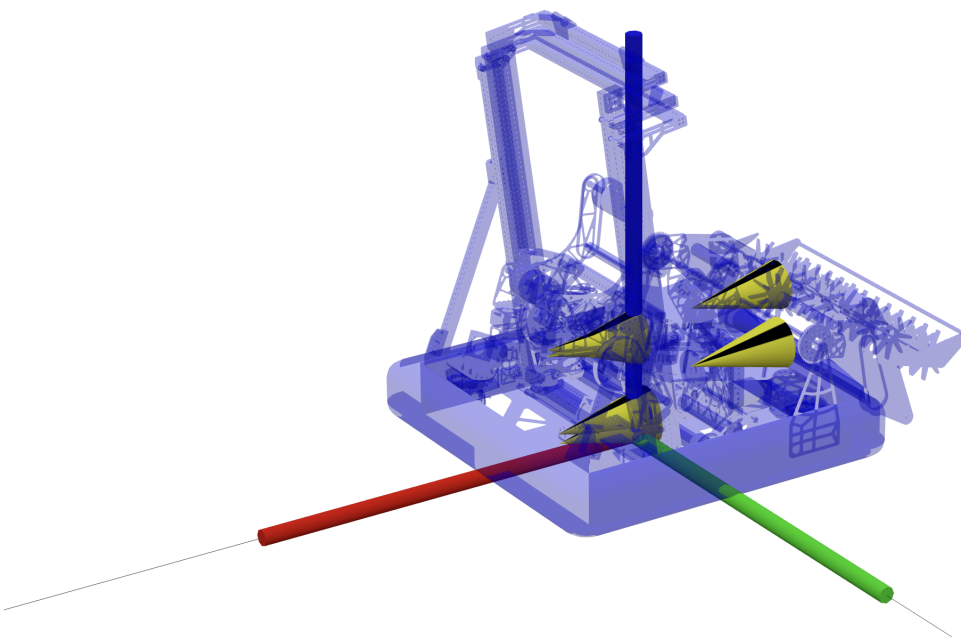


Exception in console log / 控制台日志中的异常

Log Mechanisms as 3D Components / 将机构记录为 3D 组件

AdvantageKit now offers an easier method of visualizing simple [articulated components](#) in AdvantageScope. The `LoggedMechanism2d.generate3dMechanism` method can be used to *automatically* generate a set of 3D poses for visualizing full 3D components in AdvantageScope. No need to manually transform 3D poses when creating an array of components!

AdvantageKit 现在提供了一种更简单的方法，可以在 AdvantageScope 中可视化简单的[关节连接组件](#)。`LoggedMechanism2d.generate3dMechanism` 方法可用于_自动_生成一组 3D 位姿（pose），以便在 AdvantageScope 中可视化完整的 3D 组件。创建组件数组时不再需要手动变换 3D pose！



3D mechanism components / 3D 机构组件

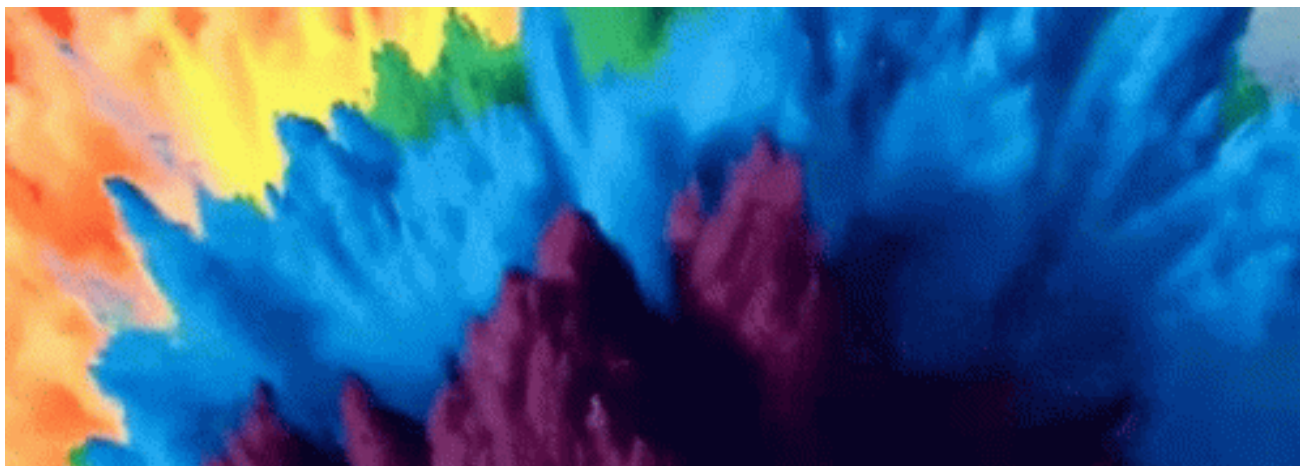
Color Logging / 颜色记录

WPILib includes a [color library](#) that can be used to simplify color operations. These objects can now be logged as inputs or outputs, and will be stored in the log as a string formatted in [Hex Triplet](#) color notation.

WPILib 包含一个可用于简化颜色操作的[颜色库](#)。这些对象现在可以作为输入或输出进行记录，并以 [Hex Triplet](#) 颜色表示法格式的字符串存储在日志中。

...

...



Rainbow / 彩虹

Online Documentation / 在线文档

The AdvantageKit documentation has been significantly improved:

AdvantageKit 文档已得到显著改进：

- Online [API documentation](#) is now available.

现在可以使用在线 [API 文档](#)。

- Page organization has been improved, including the additional of a new [theory](#) section.

页面组织方式已得到改进，包括新增的[原理](#)部分。

- Video case studies from previous FIRST Championship conference are cataloged [here](#).

往届 FIRST Championship 会议中的视频案例已整理在[此处](#)。

OVERVIEW
PACKAGE
CLASS
TREE
INDEX
HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD
DETAIL: FIELD | CONSTR | METHOD

SEARCH:

Package org.littletonrobotics.junction

Class Logger

java.lang.Object
org.littletonrobotics.junction.Logger

```

public class Logger
extends Object

```

Central class for recording and replaying log data.

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	Logger.AdvancedHooks	Advanced hooks for highly custom integration with the Logger class, including use of custom robot base classes.

Method Summary

All Methods

Static Methods

Concrete Methods

Modifier and Type	Method	Description
static void	addDataReceiver(LogDataReceiver dataReceiver)	Adds a new data receiver to process real or replayed data.
static void	disableConsoleCapture()	Disables automatic console capture.
static void	end()	Ends the logging system, including any data receivers or the replay source.
static boolean	getReceiverQueueFault()	Returns the state of the receiver queue fault.
static long	getTimestamp()	Returns the current timestamp or replayed time based on the current log entry (microseconds).
static boolean	hasReplaySource()	Returns whether a replay source is currently being used.
static void	processInputs(String key, LoggableInputs inputs)	Processes a set of inputs, logging them on the real robot or updating them in the simulator.

API documentation / API 文档

All-New Log Replay Comparison / 全新的日志回放比较

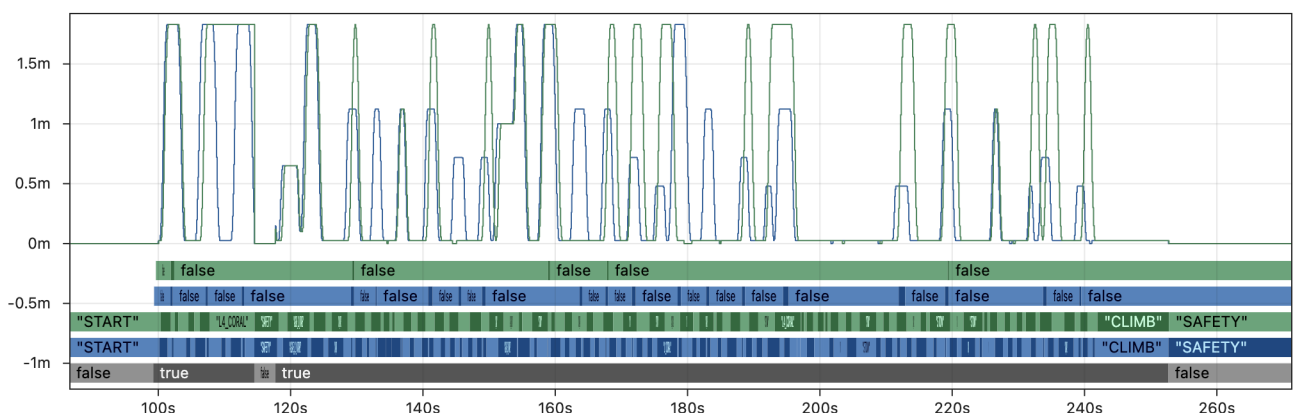
The [log replay comparison](#) page has been rewritten to be more clear, accurate, and **data-driven**. This includes deeper analysis of the effects of determinism on replay, real-world examples of the impact of rapid code iteration, and sample code exploring different subsystem code structures.

[日志回放比较](#)页面已重新编写，内容更加清晰、准确，并且以数据为依据。其中包括对确定性如何影响回放的更深入分析、快速代码迭代影响的实际案例，以及探索不同 subsystem 代码结构的示例代码。

Info / 说明

These updates bring the AdvantageKit documentation up-to-date with the [2026 release](#) of CTRE's Hoot Replay.

这些更新使 AdvantageKit 文档与 CTRE Hoot Replay 的 [2026 版本](#)保持一致。



Data on the accuracy of non-deterministic replay / 非确定性回放准确性的数据

The graph above compares data produced on a real FRC robot to data produced by non-deterministic replay. See [here](#) for details.

上图比较了真实 FRC 机器人产生的数据与非确定性回放产生的数据。详情请参阅[此处](#)。

New TalonFX(S) Swerve Template / 新的 TalonFX(S) Swerve 模板

The [TalonFX swerve template](#) has been updated with an alternative module implementation for the [TalonFXS](#) and [CANdi](#). Check the [documentation](#) for more details on configuring the template for *any* hardware setup, including a mix of devices from different vendors.

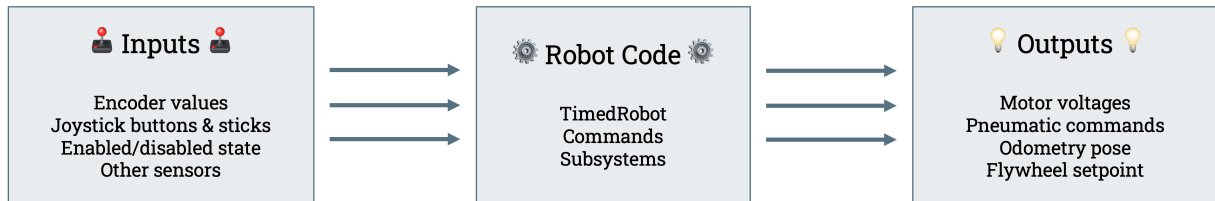
[TalonFX swerve 模板](#)已更新，增加了适用于 [TalonFXS](#) 和 [CANdi](#) 的另一种模块实现。有关为_任意_硬件配置设置模板的更多详情，包括混用不同厂商的设备，请参阅[文档](#)。

What is AdvantageKit / 什么是 AdvantageKit

What is AdvantageKit? / 什么是 AdvantageKit?

AdvantageKit is a logging framework. But to understand it, we need to start by looking at FRC robot code in general. Here's what all robot code fundamentally looks like:

AdvantageKit 是一个日志框架。但要理解它，我们需要先从总体上看看 FRC 机器人代码。所有机器人代码的基本结构如下：



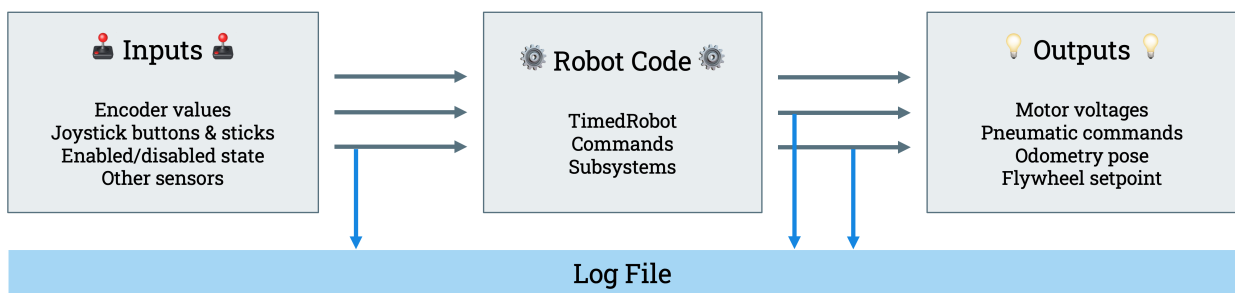
Logging Diagram #1 / 日志记录示意图 #1

There are **inputs** like sensors and driver inputs and **outputs** like motor commands and other calculated values. The task of the **robot code** is to convert **inputs** into **outputs**.

代码具有传感器数据和驾驶员输入等输入，以及电机指令和其他计算值等输出。机器人代码的任务是将输入转换为输出。

A variety of logging frameworks already exist in FRC, from built-in tools like the Driver Station and [WPILib logging](#) all the way to fully custom solutions. Most of these frameworks follow a similar structure, which looks this:

FRC 中已经存在各种日志框架，从 Driver Station 和 [WPILib 日志记录](#)等内置工具，到完全自定义的方案。大多数框架采用类似的结构，如下所示：



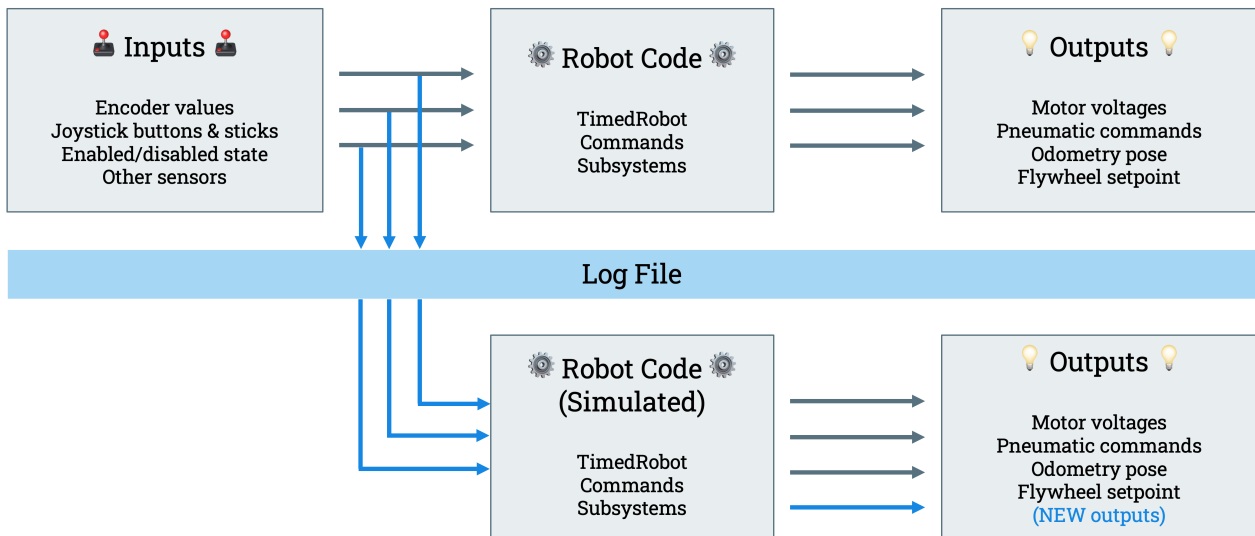
Logging Diagram #2 / 日志记录示意图 #2

A limited set of values are provided by the robot code and stored in the log. That could include sensor data, PID error, odometry pose, output commands, etc. While this has enormous value, it doesn't solve the eternal sentiment when something goes wrong: "If only we were logging one extra field!" Notice above that there is data flowing in and out of the robot code without being saved to the log file.

机器人代码提供一组有限的值，并将其存储在日志中。这些值可能包括传感器数据、PID 误差、odometry pose、输出指令等。虽然这具有很大价值，但它无法解决出问题总会出现的感叹：“要是我们多记录一个字段就好了！”注意，在上图中，有数据流入和流出机器人代码，却没有被保存到日志文件中。

AdvantageKit takes a different approach. This is what logging looks like with AdvantageKit:

AdvantageKit 采用了另一种方式。使用 AdvantageKit 时，日志记录结构如下：



Logging Diagram #3 / 日志记录示意图 #3

Instead of logging a limited set of values from the user code, AdvantageKit records *all of the data flowing into the robot code*. Every sensor value, button press, and much more is logged every loop cycle. After a match, these values can be replayed to the robot code in a simulator. Since every input command is the same, all of the internal logic of the code is replayed exactly. This allows you to log extra fields after the fact, or modify pipelines to see how they *would have* functioned during the match. This technique means that logging is more than just a tool for checking on specific issues; it's also a safety net that can be used to verify how any part of the code functions.

AdvantageKit 会记录_流入机器人代码的所有数据_，而不是只记录用户代码提供的一组有限值。每个传感器值、每次按钮按下，以及更多数据，都会在每个循环周期被记录。比赛结束后，可以在仿真器中将这值回放给机器人代码。由于每个输入指令都相同，代码中的所有内部逻辑都会被精确回放。这使你可以事后记录额外字段，或者修改 pipeline，查看它们在比赛中_本会_如何运行。这种技术意味着，日志记录不仅是检查特定问题的工具，也是一张可用于验证代码任意部分如何工作的安全网。

Tip / 提示

To see what log replay looks like in practice, check the sections below on "The Basics" or find real case studies under the "Theory" section.

要了解日志回放在实际中是什么样子，请查看下方“基础”各节，或在“原理”部分中寻找真实案例。

Championship Conference / Championship 会议

The following conference was presented by Team 6328 at the 2025 FIRST Championship, and provides an accessible overview of the capabilities and use cases of AdvantageKit.

以下会议报告由 Team 6328 在 2025 FIRST Championship 上发表，以易于理解的方式概述了 AdvantageKit 的能力和使用场景。

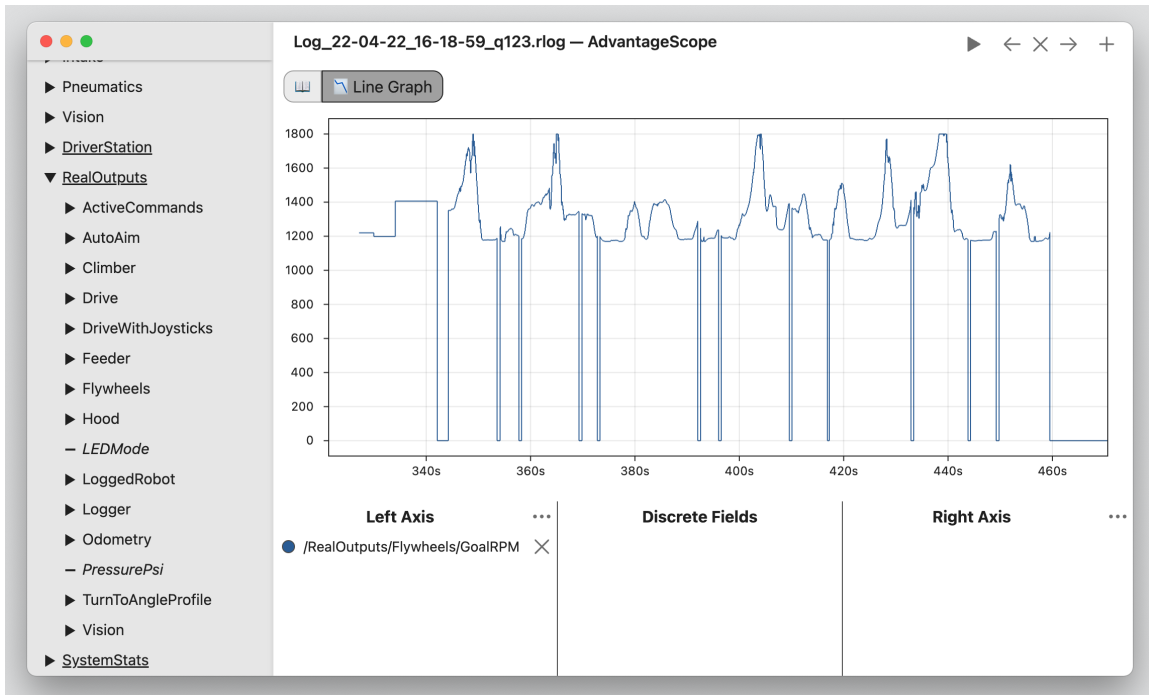
[FRC Log Replay and Simulation \(2025\) - FRC 6328 FIRST Championship Conference](#)

[FRC 日志回放与仿真（2025）— FRC 6328 FIRST Championship 会议](#)

The Basics: Output Logging / 基础：输出日志记录

Let's look at a concrete example of AdvantageKit in action. Below is a plot from 6328's 2022 robot, based on an AdvantageKit log file. It shows the flywheel's setpoint speed during a match.

我们来看一个 AdvantageKit 实际应用的具体示例。下图来自 6328 的 2022 年机器人，基于 AdvantageKit 日志文件生成。它显示了比赛期间飞轮的 setpoint 速度。



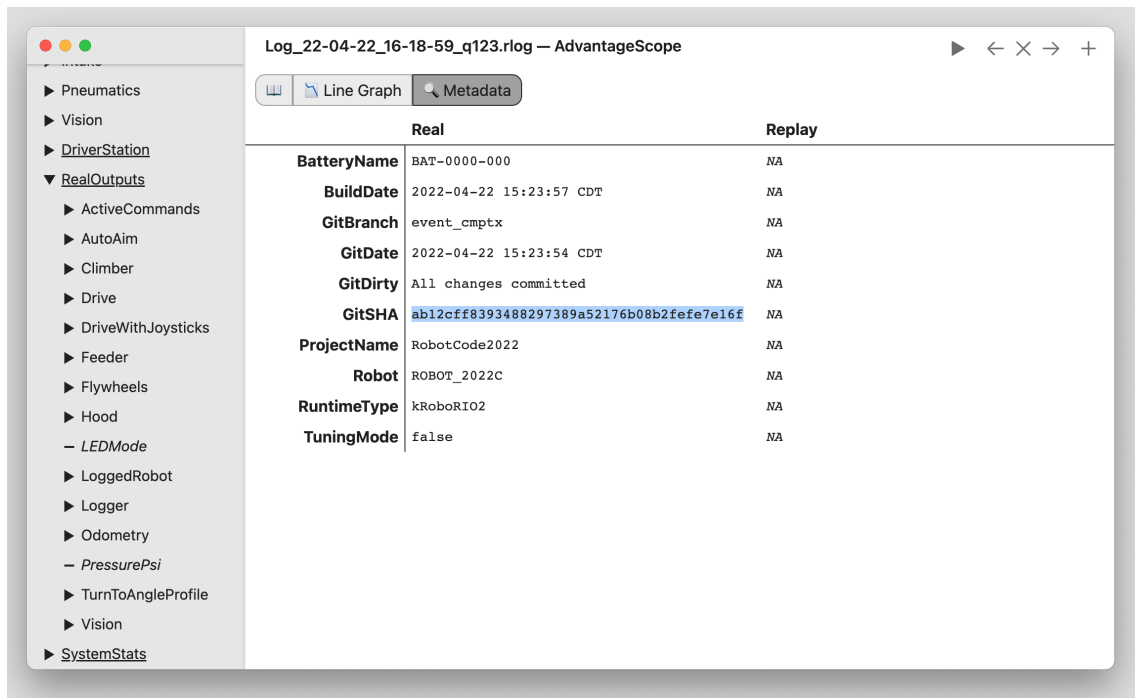
Setpoint speeds / Setpoint 速度

During teleop, this speed is based on the distance between the robot and the target. We might want to check that the flywheel speed calculation is working correctly, but this log doesn't include the calculated distance that was used. **Let's use AdvantageKit to replay the log and produce this new data.**

在 teleop 期间，该速度基于机器人与目标之间的距离。我们可能想检查飞轮速度计算是否正确，但此日志没有包含计算时使用的距离值。我们使用 **AdvantageKit** 回放日志，生成这些新数据。

First, we need to run the replay using the same version of the code that generated this log. AdvantageKit can save metadata with the log file, and we've already stored the Git hash (uniquely identifying the version of code that was running on the robot):

首先，我们需要使用生成此日志的相同代码版本运行回放。AdvantageKit 可以将元数据保存在日志文件中，我们已经存储了 Git hash（唯一标识机器人上运行的代码版本）：



Viewing metadata / 查看元数据

Next, we can return to the correct commit using git (`git checkout ab12cf...`) and add the following line to the odometry subsystem:

接下来，我们可以使用 git 返回到正确的提交（`git checkout ab12cf.....`），并在 odometry subsystem 中添加以下代码行：

```
java
```

```
Logger.recordOutput("TargetDistanceMeters", latestPose.getTranslation().getDistance(FieldConstants.hubCenter));
```

This line doesn't change the behavior of the robot code, but it logs extra "output" data every cycle. Recall that because...

这行代码不会改变机器人代码的行为，但会在每个循环中记录额外的“输出”数据。回顾一下，因为.....

- The **same code** is running on the robot and in the simulator AND...

机器人和仿真器中运行的是相同代码，并且.....

- AdvantageKit will provide the **same inputs** to the simulator that occurred on the real robot...

AdvantageKit 会向仿真器提供真实机器人上发生的相同输入.....

...the outputs from the line we added are the same as if we were logging this data on the real robot. When we start the simulator, AdvantageKit loads the log file and replays the loop cycles as quickly as possible (most laptops are much faster than the roboRIO):

.....我们添加的代码行所产生的输出，与在真实机器人上记录这些数据时完全相同。启动仿真器后，AdvantageKit 会加载日志文件，并尽可能快地回放循环周期（大多数笔记本电脑远快于 roboRIO）：

```

149 // CHECK IF ROBOT IS ALIGNED FOR LEVS
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL

buttons.run(): 0.000113s
Intake.periodic(): 0.000019s
LoggedChoosers.periodic(): 0.000010s
Hood.periodic(): 0.014214s
Drive.periodic(): 0.000076s
Feeder.periodic(): 0.000323s
Vision.periodic(): 0.000021s
Pneumatics.periodic(): 0.010482s
Climber.periodic(): 0.000027s

Warning at edu.wpi.first.wpilibj.Tracer.lambda$printEpochs$0(Tracer.java:70): SmartDashboard.updateValues(): 0.000053s
robotPeriodic(): 0.031038s
LiveWindow.updateValues(): 0.002187s
Shuffleboard.update(): 0.000004s
disabledPeriodic(): 0.000005s
simulationPeriodic(): 0.000006s

CommandScheduler loop overrun
CommandScheduler loop overrun
CommandScheduler loop overrun

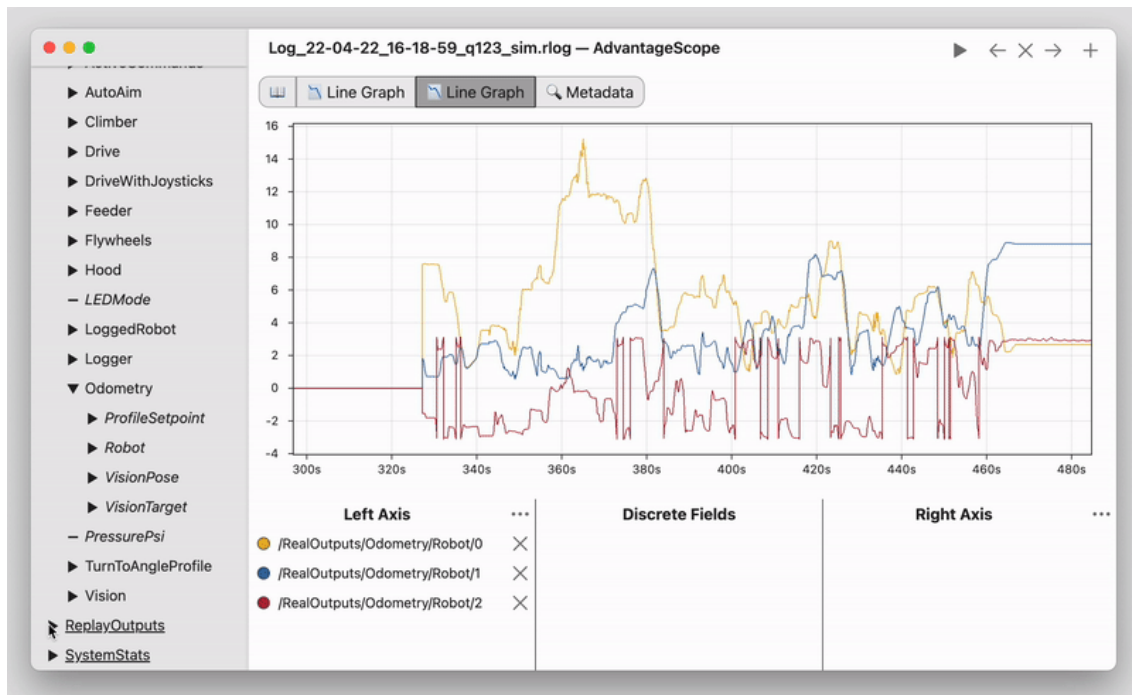
2) Ln 139, Col 51 Spaces: 2 UTF-8 LF Java WPILib

```

Log replay / 日志回放

AdvantageKit produced a new log file, which includes all of the data from the original plus a new "ReplayOutputs" table. All of these outputs match their counterparts in "RealOutputs" (logged by the robot on the field). For example, the odometry data (robot pose) was identical in replay:

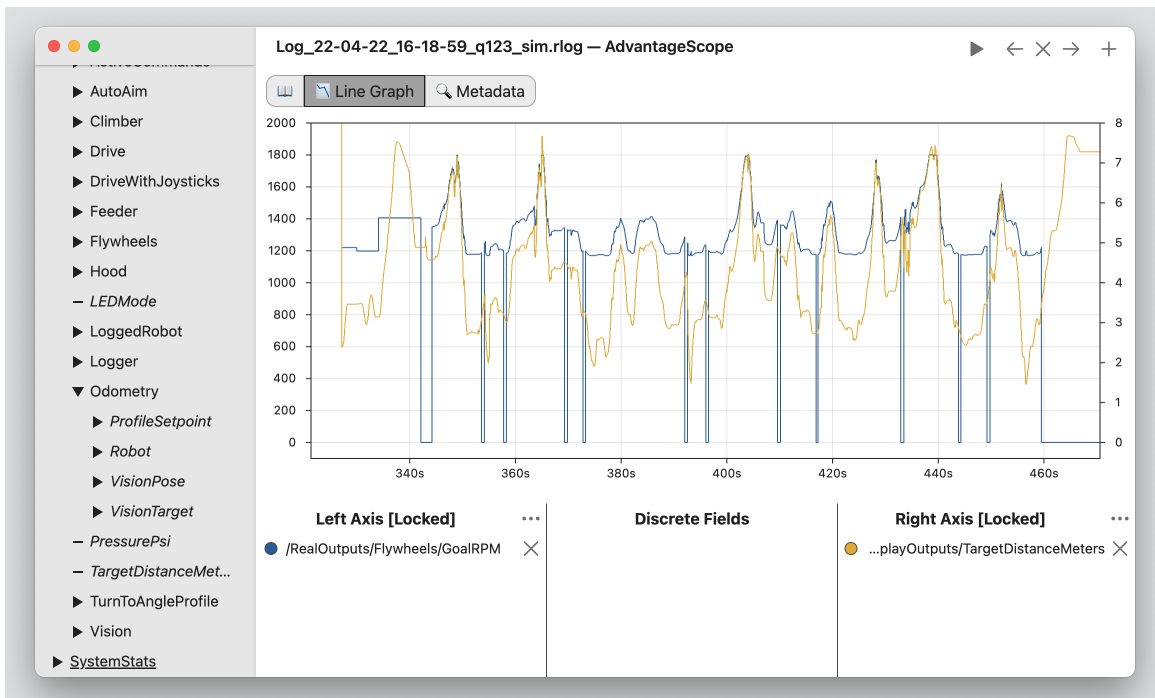
AdvantageKit 生成了一个新日志文件，其中包含原始日志的全部数据，以及一个新的 "ReplayOutputs" 表。这些输出全部与 "RealOutputs" 中对应的值一致（后者由场地上的机器人记录）。例如，odometry 数据（机器人 pose）在回放中完全相同：



Comparing odometry replay / 比较 odometry 回放

The "ReplayOutputs" also includes our new field "TargetDistanceMeters". We can confirm that the flywheel setpoint is increasing along with distance, so the calculation is working correctly. Using replay, we're able to make that conclusion with complete certainty even though we're *using data that was never logged by the robot on the field*.

"ReplayOutputs" 还包含我们新增的字段 "TargetDistanceMeters"。可以确认，飞轮 setpoint 随距离增加而增加，因此计算正确。通过回放，我们可以完全确定这一结论，尽管我们使用的是场地上的机器人从未记录过的数据。

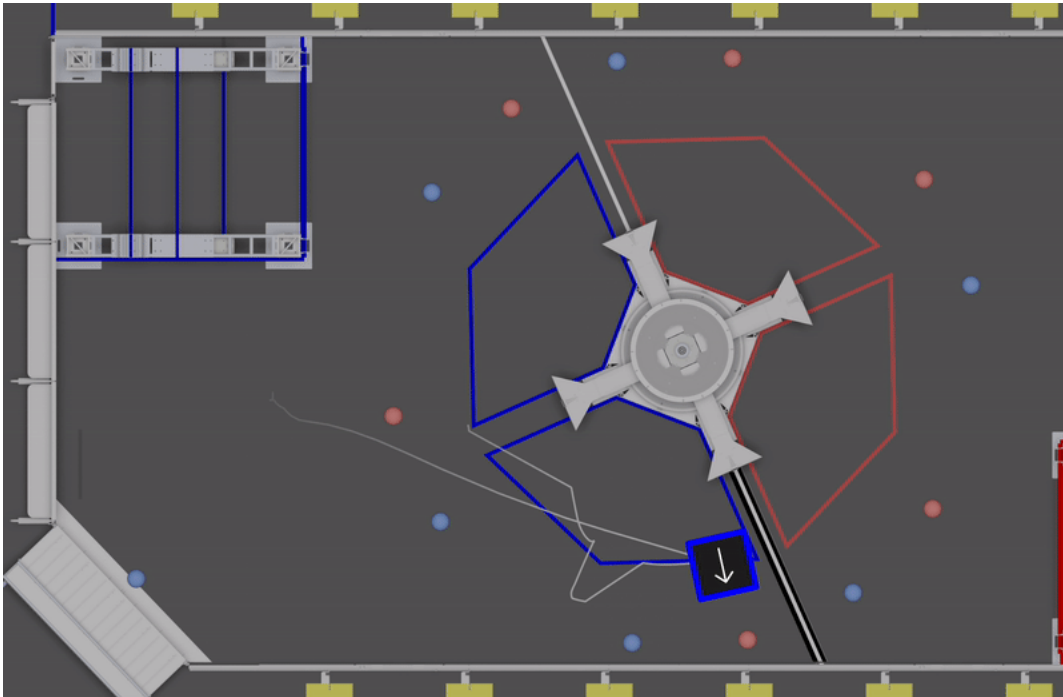


Setpoints speeds with distance / Setpoint 速度与距离

The Basics: Bug Fixes / 基础：修复缺陷

In addition to analyzing the robot code's behavior, we can also use replay to test improvements. 6328's 2022 robot used vision data from a Limelight to reset its odometry (calculated position on the field). This involves a complex pipeline in the robot code, and filtering invalid targets is critical. Below is a clip showing the robot's odometry during a particularly problematic moment. The green line indicates when vision data is being used to reset the position.

除了分析机器人代码的行为，我们还可以使用回放来测试改进。6328 的 2022 年机器人使用 Limelight 的视觉数据重置其里程计（odometry，即计算出的场地位置）。这涉及机器人代码中的复杂 pipeline，而过滤无效目标至关重要。下面的片段展示了机器人在一个问题尤为明显的时刻的 odometry。绿色线条表示正在使用视觉数据重置位置。



Original odometry / 原始 odometry

This movement is clearly impossible. Note that the Limelight on the robot is facing backward, and couldn't possibly have detected the target at the center of the field when the position initially jumps across the field. Let's use the log data to dig deeper. The vision pipeline is based on the four detected corners of each piece of vision tape around the target. Plotting these points, we would expect to see a smooth curve like the image on the left. There are five pieces of tape with distinct corners. Instead, during this moment in the log, we see the points to the right.

这种移动显然不可能发生。注意，机器人上的 Limelight 朝向后方，因此在位置最初跨越场地跳变时，它不可能检测到场地中心的目标。我们用日志数据进一步分析。视觉 pipeline 基于目标周围每条视觉反光胶带检测到的四个角点。绘制这些点时，我们预期会看到左图那样的平滑曲线。共有五条胶带，各自具有清晰的角点。然而，在日志中的这一时刻，我们看到的却是右图中的点。



Vision data / 视觉数据

The Limelight isn't seeing the vision tape, so the corners are inaccurate. In fact, many of the targets it reports don't even have four corners! To improve the filtering, let's **check that the number of reported corners is divisible by four**:

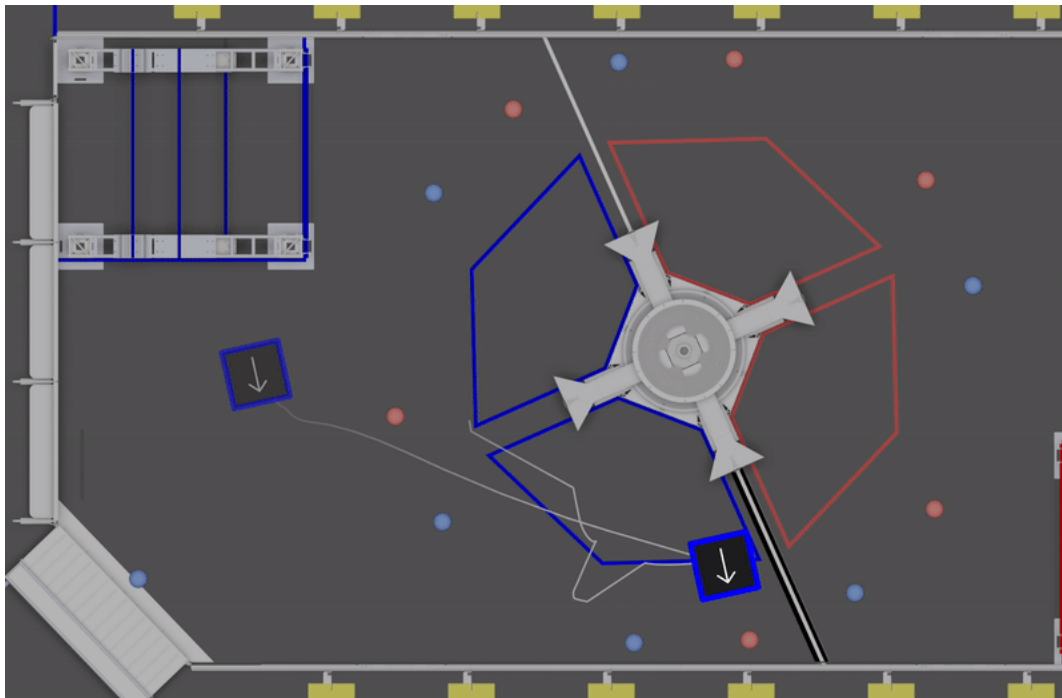
Limelight 没有看到视觉反光胶带，因此角点不准确。事实上，它报告的许多目标甚至没有四个角点！为了改善过滤，我们来检查报告的角点数量是否能被 **4** 整除：

```
java
```

```
if (inputs.cornerX.length % 4 == 0 && inputs.cornerX.length == inputs.cornerY.length) {
    // Data looks good, continue processing
} else {
    // Bad data, discard this frame
}
```

After making this change, we run the code in replay and check the "ReplayOutputs". Unlike the first example, we've modified the original code and would expect the odometry to be different in replay. Here's the same clip as before, with the translucent robot representing the robot pose in replay:

完成此更改后，我们在回放中运行代码，并检查 "ReplayOutputs"。与第一个示例不同，我们修改了原始代码，因此预期回放中的 odometry 会有所不同。下面是与之前相同的片段，其中半透明机器人表示回放中的机器人 pose：



Replayed odometry / 回放中的 odometry

The adjusted code now properly rejects the invalid target and accepts the valid one (notice that real and replay poses converge when the robot is pointed toward the real target). We can now run this code in future matches with confidence, because we know exactly what the code *would have done if this new version was running on the field*.

调整后的代码现在能够正确拒绝无效目标并接受有效目标（注意，当机器人朝向真实目标时，真实 pose 与回放 pose 会趋于一致）。现在我们可以放心地在后续比赛中运行此代码，因为我们确切知道_如果新版本在场上运行，代码会做什么_。

The Basics: Rapid Iteration / 基础：快速迭代

Some uses cases of log replay benefit from rapid iteration. One such example is tuning pose estimation algorithms. WPILib's [built-in pose estimators](#) include gains that adjust the trust in odometry and vision values. Adjusting these gains affects the stability and responsiveness of the pose estimates, but accurate testing requires real-world conditions (since the algorithm needs to be tuned to deal with the noise of real match conditions).

一些日志回放使用场景受益于快速迭代，其中一个例子是调试 pose 估计算法。WPILib 的[内置 pose 估计器](#)包含用于调整对 odometry 和视觉值信任程度的增益。调整这些增益会影响 pose 估计的稳定性和响应速度，但准确测试需要真实环境条件（因为需要调整算法以应对真实比赛环境中的噪声）。

AdvantageKit includes a feature called [replay watch](#) which is intended to address use cases that benefit from rapid iteration. The task is launched in the background, and then changes to the code are immediately replayed and displayed in AdvantageScope.

AdvantageKit 包含一个称为 [replay watch](#) 的功能，旨在满足受益于快速迭代的使用场景。任务在后台启动，随后代码更改会立即被回放，并显示在 AdvantageScope 中。

The video below demonstrates how vision standard deviations (trust values) can be tuned in replay based on the output of the pose estimator. The original outputs are shown in red, and the replayed outputs are shown in blue.

下面的视频展示了如何根据 pose 估计器的输出，在回放中调整视觉标准差（信任值）。原始输出以红色显示，回放输出以蓝色显示。

AdvantageKit Replay Watch Demo (Pose Estimation)

AdvantageKit Replay Watch 演示（pose 估计）

Note / 说明

The replay process shown in the video is in running in real-time. The replay duration depends on the size of the log file, complexity of robot project, and CPU performance of the development computer.

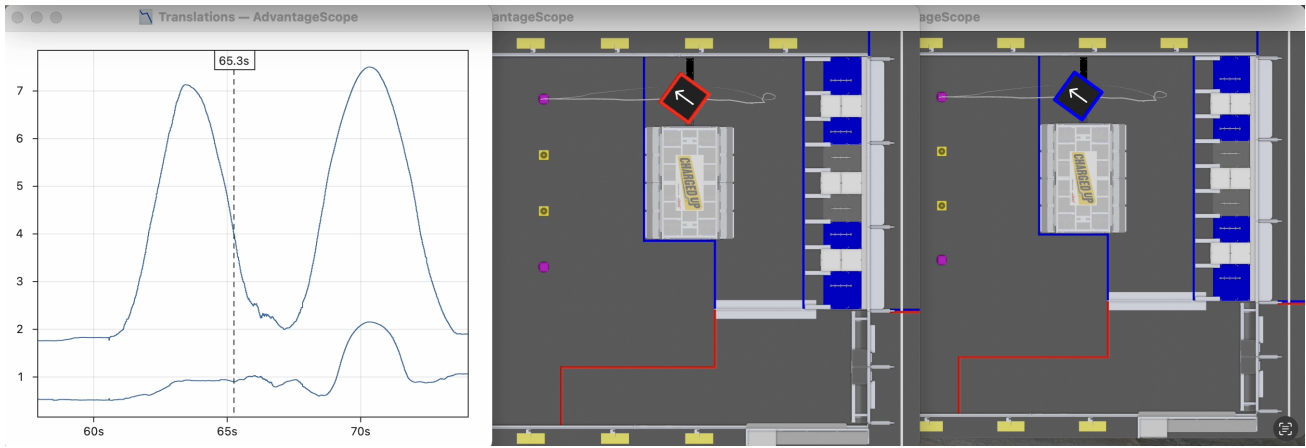
视频中展示的回放过程是实时运行的。回放时长取决于日志文件大小、机器人项目的复杂度，以及开发电脑的 CPU 性能。

Several versions of the vision gains are tested in the video:

视频中测试了几种视觉增益设置：

- **Normal trust in vision** produces identical outputs to the original pose estimates, which validates that deterministic replay is working correctly.

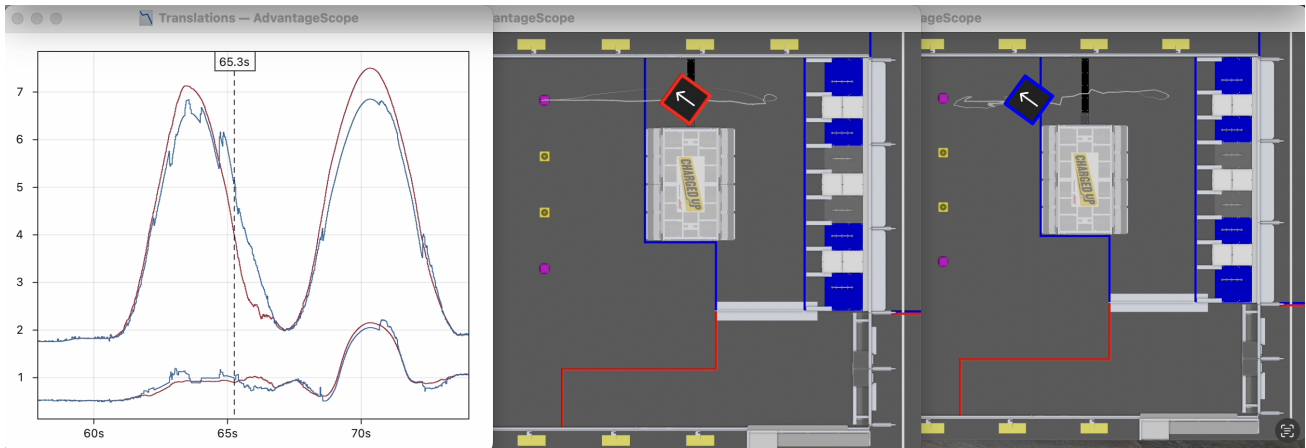
对视觉的正常信任程度产生了与原始 pose 估计完全相同的输出，验证了确定性回放正常工作。



Normal tuning / 正常调参设置

- **High trust in vision** produces more responsive but much noisier pose estimates.

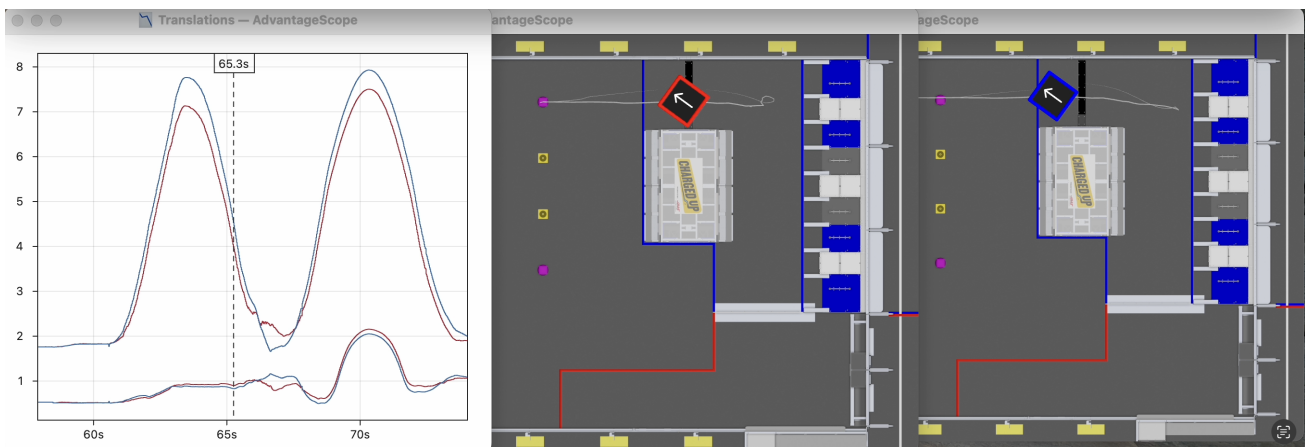
对视觉的高信任程度产生了响应更快、但噪声明显更多的 pose 估计。



High trust in vision / 对视觉的高信任程度

- **Low trust in vision** produces stabler but less responsive pose estimates, incorrectly showing the robot pass beyond the purple game piece in the middle of the field.

对视觉的低信任程度产生了更稳定、但响应较慢的 pose 估计，错误地显示机器人越过了场地中间的紫色比赛道具。



Low trust in vision / 对视觉的低信任程度

Installation / 安装

New Projects / 新项目

Info / 说明

Template projects are not currently available for the 2027 alpha versions of AdvantageKit.

目前没有适用于 AdvantageKit 2027 alpha 版本的模板项目。

For new projects, we highly recommend starting with one of the [template projects](#) attached to the [latest release](#). These projects include detailed documentation and setup instructions for many common use cases:

对于新项目，我们强烈建议从[最新发布版本](#)附带的某个[模板项目](#)开始。这些项目针对许多常见使用场景提供了详细文档和设置说明：

- **2026 KitBot Template:** For robots based on the 2026 FIRST KitBot.
2026 KitBot 模板：适用于基于 2026 FIRST KitBot 的机器人。
- **Differential Drive Template:** For other differential drive (tank) robots.
差速底盘模板：适用于其他差速（tank）底盘机器人。
- **Spark Swerve Template:** For swerve drives primarily using the Spark Max and Spark Flex, including NEO, NEO Vortex, or NEO 550 motors.
Spark Swerve 模板：适用于主要使用 Spark Max 和 Spark Flex 的 swerve drive，包括 NEO、NEO Vortex 或 NEO 550 电机。
- **TalonFX(S) Swerve Template:** For swerve drives primarily using TalonFX(S)-based motors like the Falcon 500, Kraken X60, Kraken X44, and Minion.
TalonFX(S) Swerve 模板：适用于主要使用基于 TalonFX(S) 的电机（如 Falcon 500、Kraken X60、Kraken X44 和 Minion）的 swerve drive。
- **Vision Template:** Example code for running simple vision targeting and pose estimation.
视觉模板：用于运行简单视觉瞄准和位姿（pose）估计的示例代码。
- **Skeleton Template:** Simple project with AdvantageKit installed but without subsystems or control logic.
骨架模板：已安装 AdvantageKit，但没有 subsystem 或控制逻辑的简单项目。

Existing Projects / 现有项目

Users wishing to install AdvantageKit in an existing project should check the documentation page for [existing projects](#).

希望在现有项目中安装 AdvantageKit 的用户，应查看[现有项目](#)文档页面。

Offline Installation / 离线安装

Maven artifacts for AdvantageKit can be downloaded and installed for offline use. This allows AdvantageKit to be accessed even if the Maven repository is blocked on school networks.

可以下载并安装 AdvantageKit 的 Maven 构件，以供离线使用。这样即使学校网络屏蔽了 Maven 仓库，也可以使用 AdvantageKit。

1. Download the "maven_offline.zip" asset attached to the latest [GitHub release](#).

下载最新 [GitHub 发布版本](#) 附带的 "maven_offline.zip" 文件。

2. Unzip the file into "C:\Users\Public\wpilib\YEAR\maven" on Windows or "~/wpilib/YEAR/maven" on macOS/Linux.

在 Windows 上，将文件解压到 "C:\Users\Public\wpilib\YEAR\maven"；在 macOS/Linux 上，将其解压到 "~/wpilib/YEAR/maven"。

Legacy Projects / 旧版项目

Projects based on AdvantageKit v4.0.0-beta-1 or earlier may experience build failures due to the use of an invalid GitHub Packages token. To address this issue, these releases have been republished to the current Maven repository used by v4.0.0 and later (which does not require authentication). Please follow the steps below to switch to the new Maven repository:

基于 AdvantageKit v4.0.0-beta-1 或更早版本的项目，可能因使用无效的 GitHub Packages token 而构建失败。为解决此问题，这些版本已重新发布到 v4.0.0 及后续版本使用的当前 Maven 仓库（不需要身份验证）。请按照以下步骤切换到新的 Maven 仓库：

1. Ensure that you are using the *original* vendordep JSON file (all versions of the vendordep JSON can be found on the [GitHub releases page](#)). **Do not modify this JSON file.**

确保使用的是_原始_ vendordep JSON 文件（所有版本的 vendordep JSON 都可以在 [GitHub 发布页面](#) 找到）。不要修改此 **JSON** 文件。

2. Find the block below in build.gradle which configured the GitHub Packages repository:

在 build.gradle 中找到以下用于配置 GitHub Packages 仓库的代码块：

groovy

```
repositories {
    maven {
        url = uri("https://maven.pkg.github.com/Mechanical-Advantage/AdvantageKit")
        credentials {
            username = "Mechanical-Advantage-Bot"
            password = "\u0067\u0068\u0070\u005f\u006e\u0056\u0051\u006a\u0055\u004f\u004c\u0061\u0079\u0066\u006e\u0078\u006e\u0037\u0051\u0049\u0054\u0042\u0032\u004c\u004a\u006d\u0055\u0070\u0073\u0031\u006d\u0037\u004c\u005a\u0030\u0076\u0062\u0070\u0063\u0051"
        }
    }
}
```

3. Replace that block with the new version shown below:

将该代码块替换为下面的新版本：

groovy

```
repositories {
    maven {
        url = uri("https://frcmaven.wpi.edu/artifactory/littletonrobotics-mvn-release")
    }
}
```

Existing Projects / 现有项目

To install the AdvantageKit vendordep, follow the instructions in the WPILib documentation for [installing vendor libraries](#) and choose "AdvantageKit" from the list. Alternatively, go to "WPILib: Manage Vendor Libraries" > "Install new libraries (online)" in VSCode and paste the URL below.

要安装 AdvantageKit vendordep, 请按照 WPILib 文档中的[安装 vendor library](#)说明操作, 并从列表中选择 "AdvantageKit". 也可以在 VSCode 中进入 "WPILib: Manage Vendor Libraries" > "Install new libraries (online)", 然后粘贴下面的 URL。

code

```
https://github.com/Mechanical-Advantage/AdvantageKit/releases/latest/download/AdvantageKit.json
```

Next, add the following blocks to the build.gradle file. Note that the lines under dependencies should be combined with the existing dependencies block.

接下来, 将以下代码块添加到 build.gradle 文件中。注意, dependencies 下的行应合并到现有的 dependencies 块中。

groovy

```
task(replayWatch, type: JavaExec) {
    mainClass = "org.littletonrobotics.junction.ReplayWatch"
    classpath = sourceSets.main.runtimeClasspath
}

dependencies {
    // ...
    def akitJson = new groovy.json.JsonSlurper().parseText(new File(projectDir.getAbsolutePath() + "/vendor
deps/AdvantageKit.json").text)
    annotationProcessor "org.littletonrobotics.akit:akit-autolog:$akitJson.version"
}
```

Robot Configuration / 机器人配置

The main Robot class **must inherit from** LoggedRobot (see below). LoggedRobot performs the same functions as TimedRobot, with some exceptions:

主 Robot 类必须继承 LoggedRobot (见下文)。LoggedRobot 执行与 TimedRobot 相同的功能, 但有一些例外:

- It does not support adding extra periodic functions.

它不支持添加额外的周期性函数。

- The method `setUseTiming` allows the user code to disable periodic timing and run cycles as fast as possible during replay. The timestamp read by methods like `Timer.getFPGATimestamp()` will still match the timestamp from the real robot.

`setUseTiming` 方法允许用户代码在回放期间禁用周期定时, 并尽可能快地执行循环。通过 `Timer.getFPGATimestamp()` 等方法读取的时间戳仍会与真实机器人的时间戳一致。

译注: 原文中的 `Timer.getFPGATimestamp()` 存在拼写错误; 方法名应为 `Timer.getFPGATimestamp()`。有关可确定性回放的时间戳, 参阅本地文档的 [Deterministic Timestamps](#)。

java

```
public class Robot extends LoggedRobot {
    ...
}
```

The user program is responsible for configuring and initializing the logging framework. This setup should be placed in the constructor of Robot *before any other initialization*. More information on the importance of

initializing the robot code in order can be found [here](#). An example configuration is provided below:

用户程序负责配置和初始化日志框架。此设置应放在 Robot 的构造函数中，位于_任何其他初始化之前_。有关按顺序初始化机器人代码的重要性，更多信息可在[此处](#)找到。下面提供了一个配置示例：

```
java
```

```
Logger.recordMetadata("ProjectName", "MyProject"); // Set a metadata value

if (isReal()) {
    Logger.addDataReceiver(new WPILogWriter()); // Log to a USB stick ("/U/logs")
    Logger.addDataReceiver(new NT4Publisher()); // Publish data to NetworkTables
} else {
    setUseTiming(false); // Run as fast as possible
    String logPath = LogFileUtil.findReplayLog(); // Pull the replay log from AdvantageScope (or prompt the user)
    Logger.setReplaySource(new WPILogReader(logPath)); // Read replay log
    Logger.addDataReceiver(new WPILogWriter(LogFileUtil.addPathSuffix(logPath, "_sim"))); // Save outputs to a new log
}

Logger.start(); // Start logging! No more data receivers, replay sources, or metadata values may be added.
```

Info / 说明

By default, the WPILogWriter class writes to a USB stick when running on the roboRIO. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports.**

默认情况下，在 roboRIO 上运行时，WPILogWriter 类会将数据写入 U 盘。必须将一个 **FAT32** 格式的 **U 盘** 连接到 **roboRIO** 的某个 **USB** 端口。

This setup enters replay mode for all simulator runs. If you need to run the simulator without replay (e.g. a physics simulator or Romi), extra constants or selection logic is required. See the template projects for one method of implementing this logic.

此设置会使所有仿真器运行都进入回放模式。如果需要在不回放的情况下运行仿真器（例如运行物理仿真器或 Romi），则需要额外的常量或选择逻辑。模板项目中展示了一种实现此逻辑的方法。

Version Control / 版本控制

Typically, log replay requires that the code running on the robot and in the simulator are identical. We recommend the following tools to achieve that, enabled by version control with Git.

通常，日志回放要求机器人上运行的代码与仿真器中运行的代码完全相同。我们建议使用以下工具，通过 Git 版本控制来实现这一点。

Tip / 提示

These tools are already installed in the AdvantageKit [template projects](#), but this page explains their intended usage in more detail.

这些工具已安装在 AdvantageKit 的[模板项目](#)中，但本页会更详细地解释它们的预期用法。

Gversion / Gversion

The [gversion](#) Gradle plugin produces a constants file with important metadata, including the [Git hash](#) uniquely identifying each commit. It also includes whether the tree is "dirty" (if it included uncommitted changes). The template projects include Gversion already. Otherwise, follow the installation instructions below.

[gversion](#) Gradle 插件会生成一个包含重要元数据的常量文件，其中包括唯一标识每次提交的 [Git hash](#)。它还会记录工作树是否为 "dirty"（是否包含未提交的更改）。模板项目已包含 Gversion。否则，请按照下面的安装说明操作。

Installation

安装

Add the plugin at the top of build.gradle:

在 build.gradle 顶部添加该插件：

groovy

```
plugins {
    // ...
    id "com.peterabeles.gversion" version "1.10"
}
```

Add the createVersionFile task as a dependency of compileJava:

将 createVersionFile 任务添加为 compileJava 的依赖：

groovy

```
project.compileJava.dependsOn(createVersionFile)
gversion {
    srcDir      = "src/main/java/"
    classPackage = "frc.robot"
    className   = "BuildConstants"
    dateFormat  = "yyyy-MM-dd HH:mm:ss z"
    timeZone    = "America/New_York" // Use preferred time zone
    indent      = "  "
}
```

You should also add the BuildConstants.java file to the repository .gitignore:

还应将 BuildConstants.java 文件添加到仓库的 .gitignore 中：

code

```
src/main/java/frc/robot/BuildConstants.java
```

Info / 说明

Git must be installed and available on the PATH to use the Gversion plugin. See [here](#).

要使用 Gversion 插件，必须安装 Git，且能通过 PATH 找到它。请参阅[此处](#)。

Metadata can be recorded in the log file as shown below:

可以按如下方式将元数据记录到日志文件中：

```
java
```

```
public Robot() {
    Logger.recordMetadata("GitSHA", BuildConstants.GIT_SHA);
    // ...

    Logger.start();
}
```

The metadata values can be viewed using AdvantageScope's [Metadata](#) tab. Running `git checkout ?????. . .` with the commit hash will return to the same version of code that was running on the robot (except for any uncommitted changes).

可以通过 AdvantageScope 的 [Metadata](#) 选项卡查看元数据值。使用提交 hash 运行 `git checkout ??????. . .`，会返回到机器人上运行的相同代码版本（未提交的更改除外）。

Event Deploy / 赛事部署

Code often changes repeatedly during competition, which would normally mean running code with uncommitted changes. This is a problem for log replay, since the version of code running in a particular match may be impossible to recreate afterwards. This can be addressed by including a Gradle task to automatically commit working changes to a temporary branch before every deploy.

比赛期间代码经常反复变化，通常会运行包含未提交更改的代码。这对日志回放来说是个问题，因为某场比赛中运行的代码版本事后可能无法重现。可以加入一个 Gradle 任务，在每次部署之前自动将工作区更改提交到临时分支，以解决此问题。

Installation / 安装

The Gradle task is preconfigured in the AdvantageKit template projects. Add the following lines to `build.gradle`:

AdvantageKit 模板项目已预先配置了该 Gradle 任务。将以下内容添加到 `build.gradle`：

groovy

```
// Create commit with working changes on event branches
task(eventDeploy) {
    doLast {
        if (project.gradle.startParameter.taskNames.any({ it.toLowerCase().contains("deploy") })) {
            def branchPrefix = "event"
            def branch = 'git branch --show-current'.execute().text.trim()
            def commitMessage = "Update at '${new Date().toString()}'"

            if (branch.startsWith(branchPrefix)) {
                exec {
                    workingDir(projectDir)
                    executable 'git'
                    args 'add', '-A'
                }
                exec {
                    workingDir(projectDir)
                    executable 'git'
                    args 'commit', '-m', commitMessage
                    ignoreExitValue = true
                }

                println "Committed to branch: '$branch'"
                println "Commit message: '$commitMessage'"
            } else {
                println "Not on an event branch, skipping commit"
            }
        } else {
            println "Not running deploy task, skipping commit"
        }
    }
}
createVersionFile.dependsOn(eventDeploy)
```

Usage / 用法

1. Before the event, create and check out a branch that starts with "event" such as "event_nhgrs". We recommend creating a new branch for each event.

赛事开始之前，创建并切换到一个以 "event" 开头的分支，例如 "event_nhgrs"。我们建议每场赛事都创建一个新分支。

2. Deploy robot code through any method supported by WPILib.

通过 WPILib 支持的任意方式部署机器人代码。

3. A commit is automatically created with all changes since the last commit before the deploy begins. The name of the commit includes the current timestamp (e.g. "Update at "1/31/2022, 8:30:00 AM").

部署开始之前，会自动创建一次提交，包含自上次提交以来的所有更改。提交名称包含当前时间戳（例如 "Update at "1/31/2022, 8:30:00 AM"）。

4. At the end of the event, the branch can be **"squashed and merged"** back to a normal development branch, keeping the Git history clean.

赛事结束时，可以将该分支**压缩并合并**回常规开发分支，以保持 Git 历史清晰。

5. When running log replay, find the commit hash in the log file metadata and run `git checkout` as described in the previous section. This will return to the exact version of code running on the robot (even if the commits were later squashed and merged). Since all of the changes were committed before each deploy, the simulated code is guaranteed to be identical to the original robot code.

运行日志回放时，在日志文件元数据中找到提交 hash，并按上一节所述运行 `git checkout`。这会返回到机器人上运行的确切代码版本（即使这些提交后来被压缩并合并）。由于所有更改都在每次部署前提交，因此可以保证仿真代码与原始机器人代码完全相同。

Welcome to AdvantageKit / 欢迎使用 AdvantageKit

AdvantageKit is a logging, telemetry, and replay framework developed by Team 6328. AdvantageKit enables log replay, where the full state of the robot code can be replayed in simulation based on a log file ([What is AdvantageKit?](#)).

AdvantageKit 是由 Team 6328 开发的日志记录、遥测和回放框架。AdvantageKit 实现了日志回放，可以基于日志文件在仿真中回放机器人代码的完整状态（[什么是 AdvantageKit?](#)）。

Danger / 危险

AdvantageKit is not a general-purpose logging framework. Before continuing, check the documentation for:

AdvantageKit 不是通用日志框架。继续之前，请查看以下文档：

- [AdvantageScope](#), our robot telemetry application which *does not require AdvantageKit to use*.

[AdvantageScope](#): 我们的机器人遥测应用，使用它不需要 *AdvantageKit*。

- [WPILib Data Logging](#), a simpler logging system included in WPILib (does not support log replay in simulation, but covers the needs of most teams).

[WPILib 数据日志](#): WPILib 内置的更简单的日志系统（不支持在仿真中回放日志，但能满足大多数队伍的需求）。

REQUIRED / 必需步骤

The steps below are required to complete a valid installation of AdvantageKit.

必须完成以下步骤，才能正确安装 **AdvantageKit**。

Tip / 提示

Instead of installing AdvantageKit in an existing project, consider using one of AdvantageKit's [template projects](#) to get started more quickly. These projects can be downloaded from the [latest release](#) and include prebuilt subsystems for **swerve drives**, **vision systems**, the **FIRST KitBot**, and more!

可以考虑使用 AdvantageKit 的某个[模板项目](#)来更快地开始开发，而不是在现有项目中安装 AdvantageKit。这些项目可从[最新发布版本](#)下载，包含用于 **swerve drive**、视觉系统、**FIRST KitBot** 等的预先实现的 subsystem！

1. Add the following blocks to the build.gradle file. Note that the lines under dependencies should be combined with the existing dependencies block.

将以下代码块添加到 build.gradle 文件中。注意，dependencies 下的行应合并到现有的 dependencies 块中。

groovy

```
task(replayWatch, type: JavaExec) {
    mainClass = "org.littletonrobotics.junction.ReplayWatch"
    classpath = sourceSets.main.runtimeClasspath
}

dependencies {
    // ...
    def akitJson = new groovy.json.JsonSlurper().parseText(new File(projectDir.getAbsolutePath() + "/vendor
deps/AdvantageKit.json").text)
    annotationProcessor "org.littletonrobotics.akit:akit-autolog:$akitJson.version"
}
```

2. Configure the main Robot class to inherit from LoggedRobot as shown below. See [here](#) for details.

按下方所示配置主 Robot 类，使其继承 LoggedRobot。详情请参阅[此处](#)。

```
java
```

```
public class Robot extends LoggedRobot {
    ...
}
```

3. Initialize the logging framework in the constructor of Robot *before any other initialization*. An example configuration is provided below:

在 Robot 构造函数中，位于_任何其他初始化之前_初始化日志框架。下面提供了一个配置示例：

```
java
```

```
Logger.recordMetadata("ProjectName", "MyProject"); // Set a metadata value

if (isReal()) {
    Logger.addDataReceiver(new WPILOGWriter()); // Log to a USB stick ("/U/logs")
    Logger.addDataReceiver(new NT4Publisher()); // Publish data to NetworkTables
} else {
    setUseTiming(false); // Run as fast as possible
    String logPath = LogFileUtil.findReplayLog(); // Pull the replay log from AdvantageScope (or prompt the user)
    Logger.setReplaySource(new WPILOGReader(logPath)); // Read replay log
    Logger.addDataReceiver(new WPILOGWriter(LogFileUtil.addPathSuffix(logPath, "_sim"))); // Save outputs to a new log
}

Logger.start(); // Start logging! No more data receivers, replay sources, or metadata values may be added.
```

Info / 说明

By default, the WPILOGWriter class writes to a USB stick when running on the roboRIO. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports.**

默认情况下，在 roboRIO 上运行时，WPILOGWriter 类会将数据写入 U 盘。必须将一个 **FAT32** 格式的 **U 盘** 连接到 **roboRIO** 的某个 **USB** 端口。

This setup enters replay mode for all simulator runs. If you need to run the simulator without replay (e.g. a physics simulator or Romi), extra constants or selection logic is required. See the template projects for one method of implementing this logic.

此设置会使所有仿真器运行都进入回放模式。如果需要在不回放的情况下运行仿真器（例如运行物理仿真器或 Romi），则需要额外的常量或选择逻辑。模板项目中展示了一种实现此逻辑的方法。

Note / 说明

For all support requests, please email software@team6328.org.

所有支持请求请发送邮件至 software@team6328.org。

2026 KitBot Template / 2026 KitBot 模板

The 2026 KitBot template is designed for robots based on the design of the 2026 [FIRST KitBot](#). It includes all of the features of the [differential drive template](#), along with year-specific subsystems and a simple autonomous routine. It supports a wide variety of hardware, including Spark Max/Flex, Talon SRX, and TalonFX controllers along with the navX, Pigeon 2, and similar gyros.

2026 KitBot 模板适用于基于 2026 [FIRST KitBot](#) 设计的机器人。它包含[差速底盘模板](#)的全部功能，以及该赛季特有的 subsystem 和一个简单的自动程序。它支持多种硬件，包括 Spark Max/Flex、Talon SRX 和 TalonFX 控制器，以及 navX、Pigeon 2 和类似陀螺仪。

Info / 说明
The AdvantageKit 2026 KitBot template is open-source and fully customizable :
AdvantageKit 2026 KitBot 模板是开源且完全可自定义的：
• No black boxes: Users can view and adjust all layers of the drive and launcher control stack. 没有黑箱：用户可以查看和调整底盘与发射机构控制栈的所有层。 • Customizable: IO implementations can be adjusted to support any hardware configuration (see the customization section). 可自定义：可以调整 IO 实现以支持任意硬件配置（参阅自定义部分）。 • Replayable: Every aspect of the drive control logic, launcher control, etc. can be replayed and logged in simulation using AdvantageKit's deterministic replay features with <i>guaranteed accuracy</i>. 可回放：底盘控制逻辑、发射机构控制等的每个方面，都可以使用 AdvantageKit 的确定性回放功能在仿真中回放并记录，且_保证准确性_。

Setup / 设置

1. Download the 2026 KitBot template project from the [AdvantageKit release](#) on GitHub and open it in VSCode.

从 GitHub 上的 [AdvantageKit 发布版本](#) 下载 2026 KitBot 模板项目，并在 VSCode 中打开。

2. Set up the drive subsystem using the instructions found [here](#).

按照[此处](#)的说明设置底盘 subsystem。

3. In the constructor of RobotContainer, switch the IO implementation instantiated for other subsystems based on your chosen hardware. The default is the Talon SRX.

在 RobotContainer 构造函数中，根据所选硬件切换为其他 subsystem 实例化的 IO 实现。默认使用 Talon SRX。

4. In RollerConstants, update the device CAN ID to the correct CAN ID of the motor controller (as configured in Phoenix Tuner or REV Hardware Client)

在 RollerConstants 中，将设备 CAN ID 更新为电机控制器的正确 CAN ID（在 Phoenix Tuner 或 REV Hardware Client 中配置的值）。

Differential Drive Template / 差速底盘模板

The differential drive template is designed for drives based on Spark Max, Talon FX, or Talon SRX controllers and a NavX, Pigeon 2, or similar gyro. Some of the key features of the template include:

差速底盘模板面向使用 Spark Max、Talon FX 或 Talon SRX 控制器，以及 NavX、Pigeon 2 或类似陀螺仪的底盘。模板的一些主要功能包括：

- On-controller feedback loops
在控制器上运行的反馈循环
- Physics simulation
物理仿真
- Automated characterization routines
自动特性测定流程
- Pose estimator integration (not including vision)
pose 估计器集成（不包含视觉）
- **Deterministic replay with a guarantee of accuracy**
确定性回放，并保证准确性

Info / 说明

The AdvantageKit differential drive template is **open-source** and **fully customizable**:

AdvantageKit 差速底盘模板是开源且完全可自定义的：

- **No black boxes:** Users can view and adjust all layers of the drive control stack.
没有黑箱：用户可以查看和调整底盘控制栈的所有层。
- **Customizable:** IO implementations can be adjusted to support any hardware configuration (see the [customization](#) section).
可自定义：可以调整 IO 实现以支持任意硬件配置（参阅[自定义](#)部分）。
- **Replayable:** Every aspect of the drive control logic, pose estimator, etc. can be replayed and logged in simulation using AdvantageKit's deterministic replay features with *guaranteed accuracy*.
可回放：底盘控制逻辑、pose 估计器等每个方面，都可以使用 AdvantageKit 的确定性回放功能在仿真中回放并记录，且保证准确性。

Setup / 设置

1. Download the differential drive template project from the [AdvantageKit release](#) on GitHub and open it in VSCode.
从 GitHub 上的 [AdvantageKit 发布版本](#) 下载差速底盘模板项目，并在 VSCode 中打开。
2. Click the WPILib icon in the VSCode toolbar and find the task WPILib: Set Team Number. Enter your team number and press enter.
点击 VSCode 工具栏中的 WPILib 图标，找到任务 WPILib: Set Team Number。输入你的队号并按 Enter。
3. If not already available, download and install [Git](#).
如果尚未安装，请下载并安装 [Git](#)。

4. If the project will run **only on the roboRIO 2**, uncomment lines 39-42 of `build.gradle`. These contain additional [garbage collection](#) optimizations for the RIO 2 to improve performance.

如果项目只会在 **roboRIO 2** 上运行，请取消 `build.gradle` 第 39-42 行的注释。这些行包含针对 RIO 2 的额外[垃圾回收优化](#)，用于改善性能。

5. Navigate to `src/main/java/frc/robot/subsystems/drive/DriveConstants.java` in the AdvantageKit project.

在 AdvantageKit 项目中打开 `src/main/java/frc/robot/subsystems/drive/DriveConstants.java`。

6. Update the value of `motorReduction` based on the robot's gearing. These values represent reductions and should generally be greater than one.

根据机器人的传动比更新 `motorReduction` 的值。这些值表示减速比，通常应大于 1。

7. Update the value of `trackWidth` based on the distance between the left and right sets of wheels.

根据左右两组车轮之间的距离更新 `trackWidth` 的值。

8. Update the value of `wheelRadiusMeters` to the theoretical radius of each wheel. This value can be further refined as described in the "Tuning" section below.

将 `wheelRadiusMeters` 更新为每个车轮的理论半径。可以按照下文“调参”部分的说明进一步精确测定此值。

9. Update the value of `maxSpeedMetersPerSec` to the theoretical max speed of the robot. This value can be further refined as described in the "Tuning" section below.

将 `maxSpeedMetersPerSec` 更新为机器人的理论最大速度。可以按照下文“调参”部分的说明进一步精确测定此值。

10. Set the value of `pigeonCanId` to the correct CAN ID of the Pigeon 2 (as configured using Tuner X). **If using a NavX instead of a Pigeon 2, see the [customization](#) section below.**

将 `pigeonCanId` 设置为 Pigeon 2 的正确 CAN ID（通过 Tuner X 配置的值）。如果使用 **NavX** 而非 **Pigeon 2**，请参阅下文的[自定义](#)部分。

11. Set values of the left and right leader and follower motors to the correct CAN IDs of the drive controllers (as configured in Phoenix Tuner or REV Hardware Client).

将左右两侧主电机和跟随电机的值设置为底盘控制器的正确 CAN ID（在 Phoenix Tuner 或 REV Hardware Client 中配置的值）。

12. In the constructor of `RobotContainer`, switch the IO implementations instantiated for the drive based on your chosen hardware. The default is the Talon SRX and Pigeon 2.

在 `RobotContainer` 构造函数中，根据所选硬件切换为底盘实例化的 IO 实现。默认使用 Talon SRX 和 Pigeon 2。

13. Deploy the project to the robot and connect using AdvantageScope.

将项目部署到机器人，并使用 AdvantageScope 连接。

14. Check that there are no dashboard alerts or errors in the Driver Station console. If any errors appear, verify that CAN IDs, firmware versions, and configurations of all devices.

检查 dashboard 上没有警报，且 Driver Station 控制台没有错误。如果出现任何错误，请核对所有设备的 CAN ID、固件版本和配置。

Warning / 警告

The project is configured to save log files when running on a real robot. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports to save log files.**

该项目已配置为在真实机器人上运行时保存日志文件。要保存日志文件，必须将一个 **FAT32** 格式的 U 盘连接到 roboRIO 的某个 **USB** 端口。

14. Manually rotate each side of the drive and view the position in AdvantageScope (/Drive/Module.../DrivePositionRad). Verify that the units visible in AdvantageScope (radians) match the physical motion of the module, and that positive motion corresponds to forward movement of the robot. If necessary, change the value of motorReduction, leftInverted, or rightInverted.

手动转动底盘的每一侧，并在 AdvantageScope 中查看位置 (/Drive/Module.../DrivePositionRad)。验证 AdvantageScope 中显示的单位（弧度）与模块的实际运动相符，并且正向运动对应机器人向前移动。如有必要，修改 motorReduction、leftInverted 或 rightInverted 的值。

译注：原文中的 rightInverted 应为 rightInverted，本地模板的 DriveConstants.java 使用后者。原文位置键 /Drive/Module.../DrivePositionRad 也沿用了 swerve 写法；差速底盘的位置键在下文列为 /Drive/LeftPositionRad 和 /Drive/RightPositionRad。

Tuning / 调参

Feedforward Characterization / 前馈特性测定

The project includes default **feedforward gains** for velocity control (ks and kv).

项目包含用于速度控制的默认**前馈（feedforward）增益**（ks 和 kv）。

The project includes a simple feedforward routine that can be used to quickly measure the drive ks and kv values without requiring **SysId**:

项目包含一个简单的前馈测定流程，可以快速测量底盘的 ks 和 kv 值，无需使用 **SysId**：

1. Place the robot in an open space.

将机器人放在开阔空间中。

2. Select the "Drive Simple FF Characterization" auto routine.

选择 "Drive Simple FF Characterization" 自动程序。

3. Enable the robot in autonomous. The robot will slowly accelerate forwards, similar to a SysId quasistatic test.

在 autonomous 模式下启用机器人。机器人会缓慢向前加速，类似于 SysId 准静态测试。

4. Disable the robot after at least ~5-10 seconds.

经过至少约 5-10 秒后，禁用机器人。

5. Check the console output for the measured ks and kv values, and copy them to the realKs and realKv constants in DriveConstants.java.

在控制台输出中查看测得的 ks 和 kv 值，并将它们复制到 DriveConstants.java 中的 realKs 和 realKv 常量。

Info / 说明

The feedforward model used in simulation can be characterized using the same method. **Simulation gains are stored in the simKs and simKv constants.**

仿真中使用的前馈模型可以用相同方法测定。仿真增益存储在 simKs 和 simKv 常量中。

Users who wish to characterize acceleration gains (kA) can choose to use the full [SysId](#) application. The project includes auto routines for each of the four required SysId tests. Two options are available to load data in SysId:

希望测定加速度增益（kA）的用户，可以选择使用完整的 [SysId](#) 应用。项目为所需的四项 SysId 测试分别提供了自动程序。有两种方式可将数据加载到 SysId：

译注：原文写“两种方式”，实际列出了三种数据导出方式；此处保留原文表述。

- For Spark users, the project is configured to use [URCL](#) by default. This data can be exported as described [here](#).

对于 Spark 用户，项目默认配置为使用 [URCL](#)。可按[此处](#)的说明导出这些数据。

- TalonFX (**not Talon SRX**) users can export the Hoot log file as described [here](#).

TalonFX（不是 **Talon SRX**）用户可以按[此处](#)的说明导出 Hoot 日志文件。

- Export the AdvantageKit log file as described [here](#).

按[此处](#)的说明导出 AdvantageKit 日志文件。

Wheel Radius Characterization / 车轮半径特性测定

The effective wheel radius of a robot tends to change over time as wheels are worn down, swapped, or compress into the carpet. This can have significant impacts on odometry accuracy. We recommend regularly recharacterizing wheel radius to combat these issues.

随着车轮磨损、更换或压入地毯，机器人的有效车轮半径往往会随时间变化。这可能显著影响里程计（odometry）的准确性。我们建议定期重新测定车轮半径，以应对这些问题。

We recommend the following process to measure wheel radius:

我们建议按以下流程测量车轮半径：

1. Place the robot on carpet. Characterizing on a hard floor may produce errors in the measurement, as the robot's effective wheel radius is affected by carpet compression.

将机器人放在地毯上。由于地毯压缩会影响机器人的有效车轮半径，在硬地面上测定可能造成测量误差。

2. Using AdvantageScope, record the values of `/Drive/LeftPositionRad` and `/Drive/RightPositionRad`.

使用 AdvantageScope 记录 `/Drive/LeftPositionRad` 和 `/Drive/RightPositionRad` 的值。

3. Manually push the robot directly forward as far as possible (at least 10 feet).

手动将机器人沿正前方尽可能推远（至少 10 英尺）。

4. Using a tape measure, record the linear distance traveled by the robot.

使用卷尺记录机器人移动的直线距离。

5. Record the new values of `/Drive/LeftPositionRad` and `/Drive/RightPositionRad`.

记录 `/Drive/LeftPositionRad` 和 `/Drive/RightPositionRad` 的新值。

6. The wheel radius is equal to linear distance / wheel delta in radians. The units of radius will match the units of the linear measurement.

车轮半径等于 linear distance / wheel delta in radians。半径的单位与直线距离测量值的单位相同。

Velocity PID Tuning / 速度 PID 调参

The project includes default gains for the velocity PID controller, which can be found in the "Velocity PID configuration" section of `DriveConstants.java`. These gains should be tuned for each robot.

项目包含速度 PID 控制器的默认增益，可在 `DriveConstants.java` 的 "Velocity PID configuration" 部分找到。这些增益应针对每台机器人进行调整。

Tip / 提示

More information about PID tuning can be found in the [WPILib documentation](#).

有关 PID 调参的更多信息，请参阅 [WPILib 文档](#)。

We recommend using AdvantageScope to plot the measured and setpoint values while tuning. Measured values are published to the `/Drive/LeftVelocityRadPerSec` and `/Drive/RightVelocityRadPerSec` fields and setpoint values are published to the `/RealOutputs/Drive/LeftSetpointRadPerSec` and `/RealOutputs/Drive/RightSetpointRadPerSec` fields.

我们建议在调参时使用 AdvantageScope 绘制测量值和 setpoint。测量值发布到 `/Drive/LeftVelocityRadPerSec` 和 `/Drive/RightVelocityRadPerSec` 字段，setpoint 发布到 `/RealOutputs/Drive/LeftSetpointRadPerSec` 和 `/RealOutputs/Drive/RightSetpointRadPerSec` 字段。

Info / 说明

The PID gains used in simulation can be tuned using the same method. **Simulation gains are stored separately from "real" gains in `DriveConstants.java`.**

仿真中使用的 PID 增益可以用相同方法调整。仿真增益与“真实”增益分别存储在 `DriveConstants.java` 中。

Max Speed Measurement / 最大速度测量

The effective maximum speed of a robot is typically slightly less than the theoretical max speed based on motor free speed and gearing. To ensure that the robot remains controllable at high speeds, we recommend measuring the effective maximum speed of the robot.

机器人的有效最大速度通常略低于根据电机空载转速和传动比计算的理论最大速度。为确保机器人在高速时仍可控，我们建议测量机器人的有效最大速度。

1. Set `maxSpeedMetersPerSec` in `DriveConstants.java` to the theoretical max speed of the robot based on motor free speed and gearing.

将 `DriveConstants.java` 中的 `maxSpeedMetersPerSec` 设置为根据电机空载转速和传动比计算的机器人理论最大速度。

2. Place the robot in an open space.

将机器人放在开阔空间中。

3. Plot the measured robot speed in AdvantageScope using the `/RealOutputs/Drive/LeftVelocityMetersPerSec` and `/RealOutputs/Drive/RightVelocityMetersPerSec` fields.

使用 `/RealOutputs/Drive/LeftVelocityMetersPerSec` 和 `/RealOutputs/Drive/RightVelocityMetersPerSec` 字段，在 AdvantageScope 中绘制机器人的实测速度。

4. In teleop, drive forward at full speed until the robot's velocity is no longer increasing.

在 teleop 模式下，以全速向前驾驶，直到机器人速度不再增加。

5. Record the maximum velocity achieved and update the value of `maxSpeedMetersPerSec`.

记录达到的最大速度，并更新 `maxSpeedMetersPerSec` 的值。

PathPlanner Configuration / PathPlanner 配置

The project includes a built-in configuration for [PathPlanner](#), located in the constructor of `Drive.java`. You may wish to manually adjust the robot mass, MOI, and wheel coefficient as configured at the bottom of `DriveConstants.java`

项目内置了 [PathPlanner](#) 配置，位于 `Drive.java` 的构造函数中。你可能需要手动调整 `DriveConstants.java` 底部配置的机器人质量、转动惯量（MOI）和车轮系数。

Customization / 自定义

Custom Gyro Implementations / 自定义陀螺仪实现

The project defaults to the Pigeon 2 gyro, but can be integrated with any standard gyro. An example implementation for a NavX is included.

项目默认使用 Pigeon 2 陀螺仪，但可以集成任意标准陀螺仪。项目包含一个 NavX 的示例实现。

To change the gyro implementation, switch `new GyroIOPigeon2()` in the `RobotContainer` constructor to any other implementation. For example, the `GyroIONavX` implementation is pre-configured to use a NavX connected to the MXP SPI port. See the page on [IO interfaces](#) for more details on how hardware abstraction works.

要更改陀螺仪实现，请将 `RobotContainer` 构造函数中的 `new GyroIOPigeon2()` 切换为其他实现。例如，`GyroIONavX` 实现已预先配置为使用连接到 MXP SPI 端口的 NavX。有关硬件抽象如何工作的更多详情，请参阅 [IO 接口](#) 页面。

Custom Motor Implementations / 自定义电机实现

The implementation of `ModuleIO` can be freely customized to support alternative hardware configurations, including robots without encoders. For example, the `DriveIOSpark` implementation can be customized for brushed motors by changing `MotorType.kBrushless` to `MotorType.kBrushed` and configuring the encoder counts per revolution by calling `config.encoder.countsPerRevolution(...)`.

可以自由自定义 `ModuleIO` 的实现，以支持其他硬件配置，包括没有编码器的机器人。例如，可以将 `MotorType.kBrushless` 改为 `MotorType.kBrushed`，并调用 `config.encoder.countsPerRevolution(...)` 配置编码器每转计数，从而使 `DriveIOSpark` 实现适用于有刷电机。

Vision Integration / 视觉集成

The Drive subsystem uses WPILib's [DifferentialDrivePoseEstimator](#) class for odometry updates. The subsystem exposes the `addVisionMeasurement` method to enable vision systems to publish samples.

Drive subsystem 使用 WPILib 的 [DifferentialDrivePoseEstimator](#) 类更新 odometry。该 subsystem 对外提供 `addVisionMeasurement` 方法，使视觉系统能够发布测量样本。

Tip / 提示

This project is compatible with AdvantageKit's [vision template project](#), which provides a starting point for implementing a pose estimation algorithm based on Limelight or PhotonVision.

此项目与 AdvantageKit 的[视觉模板项目](#)兼容，该模板为实现基于 Limelight 或 PhotonVision 的 pose 估计算法提供了起点。

Real-Time Thread Priority / 实时线程优先级

Optionally, the main thread can be configured to use [real-time](#) priority when running the command scheduler by removing the comments [here](#) and [here](#) (**IMPORTANT:** You must uncomment *both* lines). This may improve the consistency of loop cycle timing in some cases, but should be used with caution as it will prevent other threads from running during the user code loop cycle (including internal threads required by NetworkTables, vendors, etc).

可选地，可以通过取消[此处](#)和[此处](#)的注释，将主线程配置为在运行 Command scheduler 时使用[实时](#)优先级（重要：必须取消_两行_的注释）。在某些情况下，这可能改善循环周期定时的一致性，但应谨慎使用，因为它会阻止其他线程在用户代码循环周期内运行（包括 NetworkTables、厂商库等所需的内部线程）。

This customization **should only be used if the loop cycle time is significantly less than 20ms**, which allows other threads to continue running between user code cycles. We always recommend **thoroughly testing this change** to ensure that it does not cause unintended side effects (examples include NetworkTables lag, CAN timeouts, etc). In general, **this customization is only recommended for advanced users** who understand the potential side-effects.

只有在循环执行时间显著小于 **20ms** 时才应使用此自定义设置，这样其他线程才能在用户代码的循环之间继续运行。我们始终建议充分测试此更改，确保不会造成非预期副作用（例如 NetworkTables 延迟、CAN 超时等）。一般来说，此自定义设置仅推荐给理解其潜在副作用的高级用户。

Spark Swerve Template / Spark Swerve 模板

AdvantageKit includes two swerve project templates with built-in support for advanced features:

AdvantageKit 包含两个 swerve 项目模板，内置对高级功能的支持：

- [High-frequency odometry](#)

高频 odometry

- On-controller feedback loops

在控制器上运行的反馈循环

- Physics simulation

物理仿真

- Automated characterization routines

自动特性测定流程

- Dashboard alerts for disconnected devices

针对设备断开的 dashboard 警报

- Pose estimator integration

pose 估计器集成

- Step-by-step setup and tuning instructions with a prebuilt AdvantageScope layout

逐步设置和调参说明，以及预先配置的 AdvantageScope 布局

- **Deterministic replay** with a **guarantee of accuracy**

确定性回放，并保证准确性

By default, the Spark version of the swerve template is configured for robots with **MAXSwerve modules, four NEO Vortex drive motors, four NEO 550 turn motors, four duty cycle absolute encoders, and a NavX or Pigeon 2 gyro**. See the [TalonFX\(S\) Swerve Template](#) for swerve robots using Talon FX.

默认情况下，Spark 版 swerve 模板配置用于具有 **MAXSwerve** 模块、四个 **NEO Vortex** 驱动电机、四个 **NEO 550** 转向电机、四个占空比绝对编码器，以及 **NavX** 或 **Pigeon 2** 陀螺仪的机器人。对于使用 Talon FX 的 swerve 机器人，请参阅 [TalonFX\(S\) Swerve 模板](#)。

Info / 说明

The AdvantageKit swerve templates are **open-source** and **fully customizable**:

AdvantageKit swerve 模板是开源且完全可自定义的：

- **No black boxes:** Users can view and adjust all layers of the swerve control stack.

没有黑箱：用户可以查看和调整 swerve 控制栈的所有层。

- **Customizable:** IO implementations can be adjusted to support any hardware configuration (see the [customization](#) section).

可自定义：可以调整 IO 实现以支持任意硬件配置（参阅[自定义](#)部分）。

- **Replayable:** Every aspect of the swerve control logic, pose estimator, etc. can be replayed and logged in simulation using AdvantageKit's deterministic replay features with *guaranteed accuracy*.

可回放：swerve 控制逻辑、pose 估计器等的所有方面，都可以使用 AdvantageKit 的确定性回放功能在仿真中回放并记录，且_保证准确性_。

Setup / 设置

Tip / 提示

The swerve project folder includes a predefined AdvantageScope layout with tabs for each setup and tuning step described below. To open it, click File > Import Layout... in the tab bar of AdvantageScope and select the file AdvantageScope Swerve Calibration.json in the swerve project folder.

swerve 项目文件夹包含一个预定义的 AdvantageScope 布局，其中为下文介绍的每个设置和调参步骤提供了选项卡。要打开它，请在 AdvantageScope 的选项卡栏中点击 File > Import Layout...，并选择 swerve 项目文件夹中的 AdvantageScope Swerve Calibration.json 文件。

1. Download the Spark swerve template project from the [AdvantageKit release](#) on GitHub and open it in VSCode.

从 GitHub 上的 [AdvantageKit 发布版本](#) 下载 Spark swerve 模板项目，并在 VSCode 中打开。

2. Click the WPILib icon in the VSCode toolbar and find the task WPILib: Set Team Number. Enter your team number and press enter.

点击 VSCode 工具栏中的 WPILib 图标，找到任务 WPILib: Set Team Number。输入你的队号并按 Enter。

3. If not already available, download and install [Git](#).

如果尚未安装，请下载并安装 [Git](#)。

4. If the project will run **only on the roboRIO 2**, uncomment lines 39-42 of build.gradle. These contain additional [garbage collection](#) optimizations for the RIO 2 to improve performance.

如果项目只会在 **roboRIO 2** 上运行，请取消 build.gradle 第 39-42 行的注释。这些行包含针对 RIO 2 的额外[垃圾回收](#)优化，用于改善性能。

5. Navigate to src/main/java/frc/robot/subsystems/drive/DriveConstants.java in the AdvantageKit project.

在 AdvantageKit 项目中打开 src/main/java/frc/robot/subsystems/drive/DriveConstants.java。

6. Update the values of driveMotorReduction and turnMotorReduction based on the robot's module type and configuration. This information can typically be found on the product page for the swerve module. These values represent reductions and should generally be greater than one.

根据机器人的模块类型和配置更新 driveMotorReduction 和 turnMotorReduction 的值。通常可以在 swerve 模块的产品页面中找到这些信息。这些值表示减速比，通常应大于 1。

7. Update the values of trackWidth and wheelBase based on the distance between the left-right and front-back modules (respectively).

分别根据左右模块之间、前后模块之间的距离更新 trackWidth 和 wheelBase 的值。

8. Update the value of wheelRadiusMeters to the theoretical radius of each wheel. This value can be further refined as described in the "Tuning" section below.

将 wheelRadiusMeters 更新为每个车轮的理论半径。可以按照下文“调参”部分的说明进一步精确测定此值。

9. Update the value of maxSpeedMetersPerSec to the theoretical max speed of the robot. This value can be further refined as described in the "Tuning" section below.

将 maxSpeedMetersPerSec 更新为机器人的理论最大速度。可以按照下文“调参”部分的说明进一步精确测定此值。

10. Set the value of `pigeonCanId` to the correct CAN ID of the Pigeon 2 (as configured using Tuner X). **If using a NavX instead of a Pigeon 2, see the [customization](#) section below.**

将 `pigeonCanId` 设置为 Pigeon 2 的正确 CAN ID（通过 Tuner X 配置的值）。如果使用 **NavX** 而非 **Pigeon 2**，请参阅下文的[自定义](#)部分。

11. For each module, set the values of `...DriveMotorId` and `...TurnMotorId` to the correct CAN IDs of the drive Spark Flex and turn Spark Max (as configured in the REV Hardware Client).

对于每个模块，将 `...DriveMotorId` 和 `...TurnMotorId` 设置为驱动 Spark Flex 和转向 Spark Max 的正确 CAN ID（在 REV Hardware Client 中配置的值）。

12. For each module, set the value of `...ZeroRotation` to `new Rotation2d(0.0)`.

对于每个模块，将 `...ZeroRotation` 的值设置为 `new Rotation2d(0.0)`。

13. Deploy the project to the robot and connect using AdvantageScope.

将项目部署到机器人，并使用 AdvantageScope 连接。

14. Check that there are no dashboard alerts or errors in the Driver Station console. If any errors appear, verify that CAN IDs, firmware versions, and configurations of all devices.

检查 dashboard 上没有警报，且 Driver Station 控制台没有错误。如果出现任何错误，请核对所有设备的 CAN ID、固件版本和配置。

Warning / 警告

The project is configured to save log files when running on a real robot. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports to save log files.**

该项目已配置为在真实机器人上运行时保存日志文件。要保存日志文件，必须将一个 **FAT32** 格式的 U 盘连接到 roboRIO 的某个 USB 端口。

15. Manually rotate the turning position of each module such that the position in AdvantageScope (`/Drive/Module.../TurnPosition`) is **increasing**. The module should be rotating **counter-clockwise** as viewed from above the robot. Verify that the units visible in AdvantageScope (radians) match the physical motion of the module. If necessary, change the value of `turnInverted`, `turnEncoderInverted`, or `turnMotorReduction`.

手动转动每个模块的转向部分，使 AdvantageScope 中的位置 (`/Drive/Module.../TurnPosition`) 增加。从机器人上方看，模块应逆时针旋转。验证 AdvantageScope 中显示的单位（弧度）与模块的实际运动相符。如有必要，修改 `turnInverted`、`turnEncoderInverted` 或 `turnMotorReduction` 的值。

16. Manually rotate each drive wheel and view the position in AdvantageScope (`/Drive/Module.../DrivePositionRad`). Verify that the units visible in AdvantageScope (radians) match the physical motion of the module. If necessary, change the value of `driveMotorReduction`.

手动转动每个驱动轮，并在 AdvantageScope 中查看位置 (`/Drive/Module.../DrivePositionRad`)。验证 AdvantageScope 中显示的单位（弧度）与模块的实际运动相符。如有必要，修改 `driveMotorReduction` 的值。

17. Manually rotate each module to align it directly forward. **Verify using AdvantageScope that the drive position *increases* when the wheel rotates such that the robot would be propelled forward.** We recommend pressing a straight object such as aluminum tubing against the pairs of left and right modules to ensure accurate alignment.

手动转动每个模块，使其正对前方。使用 **AdvantageScope** 验证：当车轮沿使机器人向前推进的方向旋转时，驱动位置应_增加_。我们建议将铝管等直物体紧贴左右模块对，以确保准确对齐。

18. Record the value of `/Drive/Module.../TurnPosition` for each aligned module. Update the value of `...ZeroRotation` for each module to `new Rotation2d(<insert value>)`.

记录每个已对齐模块的 `/Drive/Module.../TurnPosition` 值。将每个模块的 `...ZeroRotation` 更新为 `new Rotation2d(<insert value>)`。

Tuning / 调参

Feedforward Characterization / 前馈特性测定

The project includes default [feedforward gains](#) for velocity control of the drive motors (kS and kV).

项目包含用于驱动电机速度控制的默认前馈（[feedforward](#)）增益（kS 和 kV）。

The project includes a simple feedforward routine that can be used to quickly measure the drive kS and kV values without requiring [SysId](#):

项目包含一个简单的前馈测定流程，可以快速测量底盘的 kS 和 kV 值，无需使用 [SysId](#)：

1. Tune turning PID gains as described [here](#).

按[此处](#)的说明调整转向 PID 增益。

2. Place the robot in an open space.

将机器人放在开阔空间中。

3. Select the "Drive Simple FF Characterization" auto routine.

选择 "Drive Simple FF Characterization" 自动程序。

4. Enable the robot in autonomous. The robot will slowly accelerate forward, similar to a SysId quasistatic test.

在 autonomous 模式下启用机器人。机器人会缓慢向前加速，类似于 SysId 准静态测试。

5. Disable the robot after at least ~5-10 seconds.

经过至少约 5-10 秒后，禁用机器人。

6. Check the console output for the measured kS and kV values, and copy them to the `driveKs` and `driveKv` constants in `DriveConstants.java`.

在控制台输出中查看测得的 kS 和 kV 值，并将它们复制到 `DriveConstants.java` 中的 `driveKs` 和 `driveKv` 常量。

Info / 说明

The feedforward model used in simulation can be characterized using the same method. **Simulation gains are stored in the `driveSimKs` and `driveSimKv` constants.**

仿真中使用的前馈模型可以用相同方法测定。仿真增益存储在 `driveSimKs` 和 `driveSimKv` 常量中。

Users who wish to characterize acceleration gains (kA) can choose to use the full [SysId](#) application. The project includes auto routines for each of the four required SysId tests. Two options are available to load data in SysId:

希望测定加速度增益（kA）的用户，可以选择使用完整的 [SysId](#) 应用。项目为所需的四项 SysId 测试分别提供了自动程序。有两种方式可将数据加载到 SysId：

译注：原文写“两种方式”，实际列出了三种数据导出方式；此处保留原文表述。

- The project is configured to use [URCL](#) by default. This data can be exported as described [here](#).

项目默认配置为使用 [URCL](#)。可按[此处](#)的说明导出这些数据。

- Export the AdvantageKit log file as described [here](#).

按[此处的说明](#)导出 AdvantageKit 日志文件。

Tip / 提示

The built-in SysId routines can be easily adapted to characterize the turn motor feedforward or the angular motion of the robot (for example, to estimate the robot's [moment of inertia](#)). The code below shows how the runCharacterization method can be adapted for these use cases.

可以方便地修改内置 SysId 流程，用于测定转向电机的前馈或机器人的角运动（例如估算机器人的[转动惯量](#)）。下面的代码展示了如何针对这些使用场景修改 runCharacterization 方法。

```
java
```

```
/** Characterize turn motor feedforward. */
public void runCharacterization(double output) {
    io.setDriveOpenLoop(0.0);
    io.setTurnOpenLoop(output);
}

/** Characterize robot angular motion. */
public void runCharacterization(double output) {
    io.setDriveOpenLoop(output);
    io.setTurnPosition(
        Rotation2d.fromDegrees(
            switch (index) {
                case 0 -> 135.0;
                case 1 -> 45.0;
                case 2 -> -135.0;
                case 3 -> -45.0;
                default -> 0.0;
            }
        ));
}
```

Wheel Radius Characterization / 车轮半径特性测定

The effective wheel radius of a robot tends to change over time as wheels are worn down, swapped, or compress into the carpet. This can have significant impacts on odometry accuracy. We recommend regularly recharacterizing wheel radius to combat these issues.

随着车轮磨损、更换或压入地毯，机器人的有效车轮半径往往会随时间变化。这可能显著影响里程计（odometry）的准确性。我们建议定期重新测定车轮半径，以应对这些问题。

The project includes an automated wheel radius characterization routine, which only requires enough space for the robot to rotate in place.

项目包含一个自动车轮半径测定流程，只需要足够让机器人原地旋转的空间。

1. Place the robot on carpet. Characterizing on a hard floor may produce errors in the measurement, as the robot's effective wheel radius is affected by carpet compression.

将机器人放在地毯上。由于地毯压缩会影响机器人的有效车轮半径，在硬地面上测定可能造成测量误差。

2. Select the "Drive Wheel Radius Characterization" auto routine.

选择 "Drive Wheel Radius Characterization" 自动程序。

3. Enable the robot in autonomous. The robot will slowly rotate in place.

在 autonomous 模式下启用机器人。机器人会缓慢原地旋转。

4. Disable the robot after at least one full rotation.

机器人至少完成一整圈旋转后，将其禁用。

5. Check the console output for the measured wheel radius, and copy the value to wheelRadiusMeters in DriveConstants.java.

在控制台输出中查看测得的车轮半径，并将该值复制到 DriveConstants.java 中的 wheelRadiusMeters。

Drive/Turn PID Tuning / 驱动 / 转向 PID 调参

The project includes default gains for the drive velocity PID controllers and turn position PID controllers, which can be found in the "Drive PID configuration" and "Turn PID configuration" sections of DriveConstants.java. These gains should be tuned for each robot.

项目包含驱动速度 PID 控制器和转向位置 PID 控制器的默认增益，可在 DriveConstants.java 的 "Drive PID configuration" 和 "Turn PID configuration" 部分找到。这些增益应针对每台机器人进行调整。

Tip / 提示

More information about PID tuning can be found in the [WPILib documentation](#).

有关 PID 调参的更多信息，请参阅 [WPILib 文档](#)。

We recommend using AdvantageScope to plot the measured and setpoint values while tuning. Measured values are published to the /RealOutputs/SwerveStates/Measured field and setpoint values are published to the /RealOutputs/SwerveStates/SetpointsOptimized field.

我们建议在调参时使用 AdvantageScope 绘制测量值和 setpoint。测量值发布到 /RealOutputs/SwerveStates/Measured 字段，setpoint 发布到 /RealOutputs/SwerveStates/SetpointsOptimized 字段。

Info / 说明

The PID gains used in simulation can be tuned using the same method. **Simulation gains are stored separately from "real" gains in DriveConstants.java.**

仿真中使用的 PID 增益可以用相同方法调整。仿真增益与“真实”增益分别存储在 DriveConstants.java 中。

Max Speed Measurement / 最大速度测量

The effective maximum speed of a robot is typically slightly less than the theoretical max speed based on motor free speed and gearing. To ensure that the robot remains controllable at high speeds, we recommend measuring the effective maximum speed of the robot.

机器人的有效最大速度通常略低于根据电机空载转速和传动比计算的理论最大速度。为确保机器人在高速时仍可控，我们建议测量机器人的有效最大速度。

1. Set maxSpeedMetersPerSec in DriveConstants.java to the theoretical max speed of the robot based on motor free speed and gearing. This value can typically be found on the product page for your chosen swerve modules.

将 DriveConstants.java 中的 maxSpeedMetersPerSec 设置为根据电机空载转速和传动比计算的机器人理论最大速度。通常可以在所选 swerve 模块的产品页面中找到此值。

2. Place the robot in a open space.

将机器人放在开阔空间中。

3. Plot the measured robot speed in AdvantageScope using the /RealOutputs/SwerveChassisSpeeds/Measured field.

使用 /RealOutputs/SwerveChassisSpeeds/Measured 字段，在 AdvantageScope 中绘制机器人的实测速度。

4. In teleop, drive forwards at full speed until the robot velocity is no longer increasing.

在 teleop 模式下，以全速向前驾驶，直到机器人速度不再增加。

5. Record the maximum velocity achieved and update the value of `maxSpeedMetersPerSec`.

记录达到的最大速度，并更新 `maxSpeedMetersPerSec` 的值。

Slip Current Measurement / 打滑电流测量

The value of `driveMotorCurrentLimit` can be tuned to avoid slipping the wheels.

可以调整 `driveMotorCurrentLimit` 的值，避免车轮打滑。

1. Place the robot against the solid wall.

将机器人紧靠坚固的墙壁放置。

2. Using AdvantageScope, plot the current of a drive motor from the `/Drive/Module.../DriveCurrentAmps` key, and the velocity of the motor from the `/Drive/Module.../DriveVelocityRadPerSec` key.

使用 AdvantageScope，通过 `/Drive/Module.../DriveCurrentAmps` 键绘制一个驱动电机的电流，并通过 `/Drive/Module.../DriveVelocityRadPerSec` 键绘制该电机的速度。

3. Accelerate forward until the drive velocity increases (the wheel slips). Note the current at this time.

向前加速，直到驱动速度增加（车轮打滑）。记录此时的电流。

4. Update the value of `driveMotorCurrentLimit` to this value.

将 `driveMotorCurrentLimit` 更新为此值。

PathPlanner Configuration / PathPlanner 配置

The project includes a built-in configuration for [PathPlanner](#), located in the constructor of `Drive.java`. You may wish to manually adjust the following values:

项目内置了 [PathPlanner](#) 配置，位于 `Drive.java` 的构造函数中。你可能需要手动调整以下值：

- Robot mass, MOI, and wheel coefficient as configured at the bottom of `DriveConstants.java`

`DriveConstants.java` 底部配置的机器人质量、转动惯量（MOI）和车轮系数。

- Drive PID constants as configured in `AutoBuilder`.

`AutoBuilder` 中配置的驱动 PID 常量。

- Turn PID constants as configured in `AutoBuilder`.

`AutoBuilder` 中配置的转向 PID 常量。

Customization / 自定义

Setting Odometry Frequency / 设置 odometry 频率

By default, the project runs at **100Hz**. This value is stored as `odometryFrequency` at the top of `DriveConstants.java` and can be changed. The project configures all devices to minimize CAN bus utilization, but we recommend monitoring utilization carefully when increasing frequency.

默认情况下，项目以 **100Hz** 运行。此值存储在 `DriveConstants.java` 顶部的 `odometryFrequency` 中，可以更改。项目会配置所有设备以尽量降低 CAN 总线利用率，但我们建议在提高频率时仔细监测总线利用率。

Switching Between Spark Max and Flex / 在 Spark Max 和 Flex 之间切换

Switching between the Spark Max and Spark Flex for drive and turn motors is very simple. In the constructor of `ModuleIOSpark`, change the call instantiating the Spark object to use `CANSparkMax` or `CANSparkFlex`. The configuration object must also be changed to the corresponding `SparkMaxConfig` or `SparkFlexConfig` class.

为驱动和转向电机在 Spark Max 与 Spark Flex 之间切换非常简单。在 `ModuleIOSpark` 构造函数中，将实例化 Spark 对象的调用改为使用 `CANSparkMax` 或 `CANSparkFlex`。配置对象也必须改为对应的 `SparkMaxConfig` 或 `SparkFlexConfig` 类。

译注：原文中的 `CANSparkMax` 和 `CANSparkFlex` 是旧版类名；本地 `ModuleIOSpark` 模板实际使用 `com.revrobotics.spark.SparkMax` 和 `com.revrobotics.spark.SparkFlex`。

When switching between motor types, the `driveGearbox` and `turnGearbox` constants in `DriveConstants` should be updated accordingly.

在电机类型之间切换时，应相应更新 `DriveConstants` 中的 `driveGearbox` 和 `turnGearbox` 常量。

Custom Gyro Implementations / 自定义陀螺仪实现

The project defaults to the Pigeon 2 gyro, but can be integrated with any standard gyro. An example implementation for a NavX is included.

项目默认使用 Pigeon 2 陀螺仪，但可以集成任意标准陀螺仪。项目包含一个 NavX 的示例实现。

To change the gyro implementation, switch `new GyroIOPigeon2()` in the `RobotContainer` constructor to any other implementation. For example, the `GyroIONavX` implementation is pre-configured to use a NavX connected to the MXP SPI port. See the page on [IO interfaces](#) for more details on how hardware abstraction works.

要更改陀螺仪实现，请将 `RobotContainer` 构造函数中的 `new GyroIOPigeon2()` 切换为其他实现。例如，`GyroIONavX` 实现已预先配置为使用连接到 MXP SPI 端口的 NavX。有关硬件抽象如何工作的更多详情，请参阅 [IO 接口](#) 页面。

The `SparkOdometryThread` class reads high-frequency gyro data for odometry alongside samples from drive encoders. This class supports both Spark devices and generic signals. Note that the gyro should be configured to publish signals at the same frequency as odometry. Call `registerSignal` with a double supplier to create a queue, as shown in the `GyroIONavX` implementation:

`SparkOdometryThread` 类会同时读取用于 odometry 的高频陀螺仪数据和驱动编码器样本。此类既支持 Spark 设备，也支持通用信号。注意，应将陀螺仪配置为以与 odometry 相同的频率发布信号。使用一个 double supplier 调用 `registerSignal` 来创建队列，如 `GyroIONavX` 实现所示：

java

```
Queue<Double> yawPositionQueue = SparkOdometryThread.getInstance().registerSignal(navX::getAngle);
```

Info / 说明

Reference the full `GyroIONavX` implementation for an example of how to create a timestamp queue and update the odometry inputs for the gyro.

有关如何创建时间戳队列并更新陀螺仪的 odometry 输入的示例，请参考完整的 `GyroIONavX` 实现。

Custom Module Implementations / 自定义模块实现

The implementation of `ModuleIOSpark` can be freely customized to support alternative hardware configurations, such as using a TalonFX(S)-based drive motor. When integrating with TalonFX(S) devices, we recommend referencing the implementation found in the `ModuleIOTalonFX` or `ModuleIOTalonFXS` classes of the [TalonFX\(S\) Swerve Template](#).

可以自由自定义 `ModuleIOSpark` 的实现，以支持其他硬件配置，例如使用基于 TalonFX(S) 的驱动电机。集成 TalonFX(S) 设备时，我们建议参考 [TalonFX\(S\) Swerve 模板](#) 中的 `ModuleIOTalonFX` 或 `ModuleIOTalonFXS` 类实现。

As described in the previous section, the `SparkOdometryThread` supports non-Spark signals through the `registerSignal` method. This allows devices from different vendors to be freely mixed.

如上一节所述，`SparkOdometryThread` 通过 `registerSignal` 方法支持非 Spark 信号，因此可以自由混用不同厂商的设备。

By default, the project uses a duty cycle encoder connected to a turn Spark Max. When using another absolute encoder (such as a CANcoder or HELIUM Canandmag), we recommend resetting the relative encoder based on the absolute encoder; the relative encoder can then be used for PID control. In this case, the following changes are required:

默认情况下，项目使用连接到转向 Spark Max 的占空比编码器。使用其他绝对编码器（如 CANcoder 或 HELIUM Canandmag）时，我们建议根据绝对编码器重置相对编码器；随后可以使用相对编码器进行 PID 控制。在这种情况下，需要进行以下更改：

1. Create the encoder object in `ModuleIOSpark` and configure it appropriately.

在 `ModuleIOSpark` 中创建编码器对象，并进行适当配置。

2. Change the feedback sensor source of the turn controller:

更改转向控制器的反馈传感器来源：

java

```
turnEncoder = turnSpark.getEncoder(); // Change the type of turnEncoder to RelativeEncoder
turnConfig.closedLoopConfig.feedbackSensor(FeedbackSensor.kPrimaryEncoder); // Was: kAbsoluteEncoder
```

3. Replace `turnConfig.absoluteEncoder...` with `turnConfig.encoder...`, and `averageDepth` with `uvwAverageDepth`. Remove the setter for `inverted(...)`.

将 `turnConfig.absoluteEncoder...` 替换为 `turnConfig.encoder...`，并将 `averageDepth` 替换为 `uvwAverageDepth`。移除 `inverted(...)` 的设置调用。

4. In the signal config for the turn motor, change all instances of `absoluteEncoderPosition...` and `absoluteEncoderVelocity...` to `primaryEncoderPosition...` and `primaryEncoderVelocity...`.

在转向电机的信号配置中，将所有 `absoluteEncoderPosition...` 和 `absoluteEncoderVelocity...` 改为 `primaryEncoderPosition...` 和 `primaryEncoderVelocity...`。

5. Incorporate the turn motor reduction in the encoder position and velocity factors:

在编码器位置和速度换算系数中计入转向电机减速比：

java

```
public static final double turnEncoderPositionFactor = 2 * Math.PI / turnMotorReduction; // Rotor Rotations
    -> Wheel Radians
public static final double turnEncoderVelocityFactor = (2 * Math.PI) / 60.0 / turnMotorReduction; // Rotor
    RPM -> Wheel Rad/Sec
```

6. Reset the relative encoder position at startup:

启动时重置相对编码器位置：

java

```
tryUntilOk(turnSpark, 5, () -> turnEncoder.setPosition(customEncoder.getPositionRadians()));
```

Vision Integration / 视觉集成

The Drive subsystem uses WPILib's `SwerveDrivePoseEstimator` class for odometry updates. The subsystem exposes the `addVisionMeasurement` method to enable vision systems to publish samples.

Drive subsystem 使用 WPILib 的 `SwerveDrivePoseEstimator` 类更新 odometry。该 subsystem 对外提供 `addVisionMeasurement` 方法，使视觉系统能够发布测量样本。

Tip / 提示

This project is compatible with AdvantageKit's [vision template project](#), which provides a starting point for implementing a pose estimation algorithm based on Limelight or PhotonVision.

此项目与 AdvantageKit 的[视觉模板项目](#)兼容，该模板为实现基于 Limelight 或 PhotonVision 的 pose 估计算法提供了起点。

Swerve Setpoint Generator / Swerve Setpoint 生成器

The project already includes basic mechanisms to reduce skidding, such as drive current limits and cosine optimization. Users who prefer more control over module skidding may wish to utilize Team 254's swerve setpoint generator. Documentation for using the version of this algorithm bundled with PathPlanner can be found [here](#). The `SwerveSetpointGenerator` should be instantiated in the Drive subsystem and used in the `runVelocity` method, as shown below:

项目已包含减少打滑的基本机制，例如驱动电流限制和余弦优化。希望更细致控制模块打滑的用户，可以使用 Team 254 的 swerve setpoint 生成器。[此处](#)提供了 PathPlanner 内附此算法版本的使用文档。应在 Drive subsystem 中实例化 `SwerveSetpointGenerator`，并在 `runVelocity` 方法中使用它，如下所示：

java

```
private final SwerveSetpointGenerator setpointGenerator;
private SwerveSetpoint previousSetpoint;

public Drive(...) {
    // ...

    setpointGenerator = new SwerveSetpointGenerator(...);
    previousSetpoint = new SwerveSetpoint(getChassisSpeeds(), getModuleStates(), DriveFeedforwards.zeroes(4));

    // ...
}

public void runVelocity(ChassisSpeeds speeds) {
    previousSetpoint = setpointGenerator.generateSetpoint(previousSetpoint, speeds, 0.02);
    SwerveModuleState[] setpointStates = previousSetpoint.moduleStates();

    // ...
}
```

译注：原文代码中的 `SwerveModuleStatep[]` 存在拼写错误；本地模板使用的类型是 `SwerveModuleState[]`。代码块按原文保留。

Advanced Physics Simulation / 高级物理仿真

The project can be easily adapted to utilize Team 5516's [maple-sim](#) library for simulation, which provides a full rigid-body simulation of the swerve drive and its interactions with the field. Check the documentation for more details on how to install and use the library.

可以方便地修改项目，使用 Team 5516 的 [maple-sim](#) 库进行仿真，该库提供 swerve drive 及其与场地交互的完整刚体仿真。有关该库的安装和使用详情，请查看其文档。

Real-Time Thread Priority / 实时线程优先级

Optionally, the main thread can be configured to use [real-time](#) priority when running the command scheduler by removing the comments [here](#) and [here](#) (**IMPORTANT:** You must uncomment *both* lines). This may improve the consistency of loop cycle timing in some cases, but should be used with caution as it will prevent other threads from running during the user code loop cycle (including internal threads required by NetworkTables, vendors, etc).

可选地，可以通过取消[此处](#)和[此处](#)的注释，将主线程配置为在运行 Command scheduler 时使用[实时](#)优先级（重要：必须取消_两行_的注释）。在某些情况下，这可能改善循环周期定时的一致性，但应谨慎使用，因为它会阻止其他线程在用户代码循环周期内运行（包括 NetworkTables、厂商库等所需的内部线程）。

This customization **should only be used if the loop cycle time is significantly less than 20ms**, which allows other threads to continue running between user code cycles. We always recommend **thoroughly testing this change** to ensure that it does not cause unintended side effects (examples include NetworkTables lag, CAN timeouts, etc). In general, **this customization is only recommended for advanced users** who understand the potential side-effects.

只有在循环执行时间显著小于 **20ms** 时才应使用此自定义设置，这样其他线程才能在用户代码的循环之间继续运行。我们始终建议充分测试此更改，确保不会造成非预期副作用（例如 NetworkTables 延迟、CAN 超时等）。一般来说，此自定义设置仅推荐给理解其潜在副作用的高级用户。

TalonFX(S) Swerve Template / TalonFX(S) Swerve 模板

AdvantageKit includes two swerve project templates with built-in support for advanced features:

AdvantageKit 包含两个 swerve 项目模板，内置对高级功能的支持：

- Easy setup with Tuner X swerve project generator

通过 Tuner X swerve 项目生成器方便地进行设置

- [High-frequency odometry](#)

高频 odometry

- CANivore time sync (for Phoenix Pro users)

CANivore 时间同步（适用于 Phoenix Pro 用户）

- On-controller feedback loops

在控制器上运行的反馈循环

- Physics simulation

物理仿真

- Automated characterization routines

自动特性测定流程

- Dashboard alerts for disconnected devices

针对设备断开的 dashboard 警报

- Pose estimator integration using standard FPGA timestamps

使用标准 FPGA 时间戳的 pose 估计器集成

- Step-by-step setup and tuning instructions with a prebuilt AdvantageScope layout

逐步设置和调参说明，以及预先配置的 AdvantageScope 布局

- **Deterministic replay with a guarantee of accuracy**

确定性回放，并保证准确性

By default, the TalonFX(S) version of the swerve template is configured for robots with **four TalonFX drive motors, four TalonFX turn motors, four CANcoders, and a NavX or Pigeon 2 gyro**. An alternative IO implementation is provided for robots with **four TalonFXS drive motors, four TalonFXS turn motors, and four PWM encoders connected to CANDi**. These implementations can be freely mixed to support [alternative hardware configurations](#). Also see the [Spark Swerve Template](#) for swerve robots using Spark Max/Flex.

默认情况下，TalonFX(S) 版 swerve 模板配置用于具有四个 **TalonFX** 驱动电机、四个 **TalonFX** 转向电机、四个 **CANcoder**，以及 **NavX** 或 **Pigeon 2** 陀螺仪的机器人。另提供一种 IO 实现，适用于具有四个 **TalonFXS** 驱动电机、四个 **TalonFXS** 转向电机，以及四个连接到 **CANDi** 的 **PWM** 编码器的机器人。这些实现可以自由混用，以支持[其他硬件配置](#)。对于使用 Spark Max/Flex 的 swerve 机器人，也请参阅 [Spark Swerve 模板](#)。

Info / 说明

The AdvantageKit swerve templates are **open-source** and **fully customizable**:

AdvantageKit swerve 模板是开源且完全可自定义的：

- **No black boxes:** Users can view and adjust all layers of the swerve control stack.

没有黑箱：用户可以查看和调整 swerve 控制栈的所有层。

- **Customizable:** IO implementations can be adjusted to support any hardware configuration (see the [customization](#) section).

可自定义：可以调整 IO 实现以支持任意硬件配置（参阅[自定义](#)部分）。

- **Replayable:** Every aspect of the swerve control logic, pose estimator, etc. can be replayed and logged in simulation using AdvantageKit's deterministic replay features with *guaranteed accuracy*.

可回放：swerve 控制逻辑、pose 估计器等的每个方面，都可以使用 AdvantageKit 的确定性回放功能在仿真中回放并记录，且_保证准确性_。

Setup / 设置

Tip / 提示

The swerve project folder includes a predefined AdvantageScope layout with tabs for each setup and tuning step described below. To open it, click File > Import Layout... in the tab bar of AdvantageScope and select the file AdvantageScope Swerve Calibration.json in the swerve project folder.

swerve 项目文件夹包含一个预定义的 AdvantageScope 布局，其中为下文介绍的每个设置和调参步骤提供了选项卡。要打开它，请在 AdvantageScope 的选项卡栏中点击 File > Import Layout...，并选择 swerve 项目文件夹中的 AdvantageScope Swerve Calibration.json 文件。

Swerve Project Generator / Swerve 项目生成器

Danger / 危险

CTRE only permits the swerve project generator to be used on swerve robots with **exclusively CTRE hardware** (including a Pigeon 2). Otherwise, switch to the "Manual" tab for standard setup instructions.

CTRE 只允许在完全使用 **CTRE** 硬件（包括 Pigeon 2）的 swerve 机器人上使用 swerve 项目生成器。其他情况请切换到“Manual”选项卡，查看标准设置说明。

1. Download the TalonFX swerve template project from the [AdvantageKit release](#) on GitHub and open it in VSCode.

从 GitHub 上的 [AdvantageKit 发布版本](#) 下载 TalonFX swerve 模板项目，并在 VSCode 中打开。

2. Click the WPILib icon in the VSCode toolbar and find the task WPILib: Set Team Number. Enter your team number and press enter.

点击 VSCode 工具栏中的 WPILib 图标，找到任务 WPILib: Set Team Number。输入你的队号并按 Enter。

3. If not already available, download and install [Git](#).

如果尚未安装，请下载并安装 [Git](#)。

4. If the project will run **only on the roboRIO 2**, uncomment lines 39-42 of build.gradle. These contain additional [garbage collection](#) optimizations for the RIO 2 to improve performance.

如果项目只会在 **roboRIO 2** 上运行，请取消 build.gradle 第 39-42 行的注释。这些行包含针对 RIO 2 的额外[垃圾回收](#)优化，用于改善性能。

5. Follow the instructions in the Phoenix documentation for the [Tuner X Swerve Project Generator](#).

按照 Phoenix 文档中 [Tuner X Swerve 项目生成器](#) 的说明操作。

- On the final screen in Tuner X, choose "Generate only TunerConstants" and overwrite the file located at `src/main/java/frc/robot/generated/TunerConstants.java`.

在 Tuner X 的最后一个界面中，选择 "Generate only TunerConstants"，并覆盖位于 `src/main/java/frc/robot/generated/TunerConstants.java` 的文件。

- In `TunerConstants.java`, comment out the [last import](#) and [last method](#). Before removing them, both lines will be marked as errors in VSCode.

在 `TunerConstants.java` 中，注释掉[最后一条 import](#) 和[最后一个方法](#)。移除之前，这两处都会在 VSCode 中标记为错误。

- In `TunerConstants.java`, change `kSteerInertia` to 0.004 and `kDriveInertia` to 0.025.

在 `TunerConstants.java` 中，将 `kSteerInertia` 改为 0.004，将 `kDriveInertia` 改为 0.025。

- If the robot does not use the default arrangement of 8 TalonFXs and 4 CANcoders, please see the section [here](#) on alternative module IO implementations.

如果机器人没有使用默认的 8 个 TalonFX 和 4 个 CANcoder 配置，请参阅[此处](#)有关其他模块 IO 实现的部分。

Warning / 警告

The project is configured to save log files when running on a real robot. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports to save log files.**

该项目已配置为在真实机器人上运行时保存日志文件。要保存日志文件，必须将一个 **FAT32** 格式的 **U** 盘连接到 **roboRIO** 的某个 **USB** 端口。

Manual / 手动设置

- Download the TalonFX swerve template project from the [AdvantageKit release](#) on GitHub and open it in VSCode.

从 GitHub 上的 [AdvantageKit 发布版本](#) 下载 TalonFX swerve 模板项目，并在 VSCode 中打开。

- Click the WPILib icon in the VSCode toolbar and find the task WPILib: Set Team Number. Enter your team number and press enter.

点击 VSCode 工具栏中的 WPILib 图标，找到任务 WPILib: Set Team Number。输入你的队号并按 Enter。

- If not already available, download and install [Git](#).

如果尚未安装，请下载并安装 [Git](#)。

- If the project will run **only on the roboRIO 2**, uncomment lines 39-42 of `build.gradle`. These contain additional [garbage collection](#) optimizations for the RIO 2 to improve performance.

如果项目只会在 **roboRIO 2** 上运行，请取消 `build.gradle` 第 39-42 行的注释。这些行包含针对 RIO 2 的额外[垃圾回收](#)优化，用于改善性能。

- Navigate to `src/main/java/frc/robot/generated/TunerConstants.java` in the AdvantageKit project.

在 AdvantageKit 项目中打开 `src/main/java/frc/robot/generated/TunerConstants.java`。

- Update the values of `kDriveGearRatio` and `kSteerGearRatio` based on the robot's module type and configuration. This information can typically be found on the product page for the swerve module. These values represent reductions and should generally be greater than one.

根据机器人的模块类型和配置更新 `kDriveGearRatio` 和 `kSteerGearRatio` 的值。通常可以在 swerve 模块的产品页面中找到这些信息。这些值表示减速比，通常应大于 1。

- Update the value of `kWheelRadius` to the theoretical radius of each wheel. This value can be further refined as described in the "Tuning" section below.

将 `kWheelRadius` 更新为每个车轮的理论半径。可以按照下文“调参”部分的说明进一步精确测定此值。

- Update the value of `kSpeedAt12Volts` to the theoretical max speed of the robot. This value can be further refined as described in the "Tuning" section below.

将 `kSpeedAt12Volts` 更新为机器人的理论最大速度。可以按照下文“调参”部分的说明进一步精确测定此值。

- Update the value of `kCANBus` based on the CAN bus used by the drive devices. Check the [CANBus API](#) documentation for details on possible values.

根据底盘设备使用的 CAN 总线更新 `kCANBus` 的值。有关可用值的详情，请参阅 [CANBus API](#) 文档。

- Set the value of `kPigeonId` to the correct CAN ID of the Pigeon 2 (as configured using Tuner X). **If using a NavX instead of a Pigeon 2, see the [customization](#) section below.**

将 `kPigeonId` 设置为 Pigeon 2 的正确 CAN ID（通过 Tuner X 配置的值）。如果使用 **NavX** 而非 **Pigeon 2**，请参阅下文的[自定义](#)部分。

- For each module, set the values of `k...DriveMotorId`, `k...SteerMotorId`, and `k...EncoderId` to the correct CAN IDs of the drive TalonFX(S), turn TalonFX(S), and CANcoder/CANdi (as configured in Tuner X).

对于每个模块，将 `k...DriveMotorId`、`k...SteerMotorId` 和 `k...EncoderId` 设置为驱动 TalonFX(S)、转向 TalonFX(S) 和 CANcoder/CANdi 的正确 CAN ID（在 Tuner X 中配置的值）。

- For each module, set the values of `k...XPos` and `k...YPos` based on the distance from each module to the center of the robot. Positive X values are closer to the front of the robot and positive Y values are closer to the left side of the robot.

对于每个模块，根据各模块相对于机器人中心的距离设置 `k...XPos` 和 `k...YPos`。正 X 值对应更靠近机器人前方的位置，正 Y 值对应更靠近机器人左侧的位置。

- For each module, set the value of `k...EncoderOffset` to `Radians.of(0.0)`.

对于每个模块，将 `k...EncoderOffset` 的值设置为 `Radians.of(0.0)`。

- Deploy the project to the robot and connect using AdvantageScope.

将项目部署到机器人，并使用 AdvantageScope 连接。

- Check that there are no dashboard alerts or errors in the Driver Station console. If any errors appear, verify the CAN IDs, firmware versions, and configurations of all devices.

检查 dashboard 上没有警报，且 Driver Station 控制台没有错误。如果出现任何错误，请核对所有设备的 CAN ID、固件版本和配置。

Warning / 警告

The project is configured to save log files when running on a real robot. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports to save log files.**

该项目已配置为在真实机器人上运行时保存日志文件。要保存日志文件，必须将一个 **FAT32** 格式的 U 盘连接到 roboRIO 的某个 USB 端口。

16. Manually rotate the turning position of each module such that the position in AdvantageScope (/Drive/Module.../TurnPosition) is **increasing**. The module should be rotating **counter-clockwise** as viewed from above the robot. Verify that the units visible in AdvantageScope (radians) match the physical motion of the module. If necessary, change the value of `k...SteerMotorInverted` or `kSteerGearRatio`.

手动转动每个模块的转向部分，使 AdvantageScope 中的位置 (/Drive/Module.../TurnPosition) 增加。从机器人上方看，模块应逆时针旋转。验证 AdvantageScope 中显示的单位（弧度）与模块的实际运动相符。如有必要，修改 `k...SteerMotorInverted` 或 `kSteerGearRatio` 的值。

17. Manually rotate each drive wheel and view the position in AdvantageScope (/Drive/Module.../DrivePositionRad). Verify that the units visible in AdvantageScope (radians) match the physical motion of the module. If necessary, change the value of `kDriveGearRatio`.

手动转动每个驱动轮，并在 AdvantageScope 中查看位置 (/Drive/Module.../DrivePositionRad)。验证 AdvantageScope 中显示的单位（弧度）与模块的实际运动相符。如有必要，修改 `kDriveGearRatio` 的值。

18. Manually rotate each module to align it directly forward. **Verify using AdvantageScope that the drive position *increases* when the wheel rotates such that the robot would be propelled forward.** We recommend pressing a straight object such as aluminum tubing against the pairs of left and right modules to ensure accurate alignment.

手动转动每个模块，使其正对前方。使用 **AdvantageScope** 验证：当车轮沿使机器人向前推进的方向旋转时，驱动位置应_增加_。我们建议将铝管等直物体紧贴左右模块对，以确保准确对齐。

19. Record the value of /Drive/Module.../TurnPosition for each aligned module. Update the value of `k...EncoderOffset` for each module to `Radians.of(<insert value>)`. **The value saved in TunerConstants must be the *negative* of the value displayed in AdvantageScope (i.e. positive values become negative and vice versa).**

记录每个已对齐模块的 /Drive/Module.../TurnPosition 值。将每个模块的 `k...EncoderOffset` 更新为 `Radians.of(<insert value>)`。保存在 TunerConstants 中的值必须是 **AdvantageScope** 显示值的_相反数_（即正值变为负值，负值变为正值）。

20. If the robot does not use the default arrangement of 8 TalonFXs and 4 CANcoders, please see the section [here](#) on alternative module IO implementations. If the robot does not use a Pigeon 2, please see the section [here](#) on alternative gyro options.

如果机器人没有使用默认的 8 个 TalonFX 和 4 个 CANcoder 配置，请参阅[此处](#)有关其他模块 IO 实现的部分。如果机器人没有使用 Pigeon 2，请参阅[此处](#)有关其他陀螺仪选项的部分。

Tuning / 调参

Torque-Current Control / 转矩电流控制

The project defaults to voltage control for both the drive and turn motors. Phoenix Pro subscribers can optionally switch to torque-current control, as described in the [Phoenix documentation](#). This can be configured by changing the values of `kSteerClosedLoopOutput` and/or `kDriveClosedLoopOutput` in `TunerConstants.java` to `ClosedLoopOutputType.TorqueCurrentFOC`.

项目默认对驱动和转向电机都使用电压控制。Phoenix Pro 订阅用户可以选择切换到转矩电流控制，具体如 [Phoenix 文档](#) 所述。可通过将 `TunerConstants.java` 中的 `kSteerClosedLoopOutput` 和 / 或 `kDriveClosedLoopOutput` 改为 `ClosedLoopOutputType.TorqueCurrentFOC` 进行配置。

Info / 说明

Torque-current control requires different gains than voltage control. We recommend following the steps below to tune feedforward and PID gains.

转矩电流控制所需的增益与电压控制不同。我们建议按照下面的步骤调整前馈和 PID 增益。

Warning / 警告

CTRE does not allow torque-current control on the TalonFXS.

CTRE 不允许在 TalonFXS 上使用转矩电流控制。

Feedforward Characterization / 前馈特性测定

The project includes default [feedforward gains](#) for velocity control of the drive motors (k_s and k_v), acceleration control of the drive motors (k_a), and velocity control of the turn motors (k_s and k_v).

项目包含用于驱动电机速度控制（ k_s 和 k_v ）、驱动电机加速度控制（ k_a ），以及转向电机速度控制（ k_s 和 k_v ）的默认[前馈（feedforward）增益](#)。

Info / 说明

The AdvantageKit template requires different feedforward gains than CTRE's default swerve code, because it applies the swerve gear ratio using the TalonFX(S) firmware and not on the RIO.

AdvantageKit 模板所需的前馈增益与 CTRE 默认 swerve 代码不同，因为它通过 TalonFX(S) 固件应用 swerve 减速比，而不是在 RIO 上应用。

Tip / 提示

The drive k_s and k_v gains should **always** be characterized (as described below). The drive/turn k_a gains and turn k_s and k_v gains are unnecessary in most cases, but can be tuned by advanced users.

驱动 k_s 和 k_v 增益始终应进行特性测定（如下所述）。驱动 / 转向的 k_a 增益，以及转向的 k_s 和 k_v 增益，在大多数情况下并不需要，但高级用户可以进行调整。

The project includes a simple feedforward routine that can be used to quickly measure the drive k_s and k_v values without requiring [SysId](#):

项目包含一个简单的前馈测定流程，可以快速测量底盘的 k_s 和 k_v 值，无需使用 [SysId](#)：

1. Tune turning PID gains as described [here](#).

按[此处](#)的说明调整转向 PID 增益。

2. Place the robot in an open space.

将机器人放在开阔空间中。

3. Select the "Drive Simple FF Characterization" auto routine.

选择 "Drive Simple FF Characterization" 自动程序。

4. Enable the robot in autonomous. The robot will slowly accelerate forwards, similar to a SysId quasistatic test.

在 autonomous 模式下启用机器人。机器人会缓慢向前加速，类似于 SysId 准静态测试。

5. Disable the robot after at least ~5-10 seconds.

经过至少约 5-10 秒后，禁用机器人。

- Check the console output for the measured k_s and k_v values, and copy them to the driveGains config in `TunerConstants.java`.

在控制台输出中查看测得的 k_s 和 k_v 值，并将它们复制到 `TunerConstants.java` 中的 `driveGains` 配置。

Info / 说明

The feedforward model used in simulation can be characterized using the same method. **Simulation gains are stored in `ModuleIOSim.java` instead of `TunerConstants.java`.**

仿真中使用的前馈模型可以用相同方法测定。仿真增益存储在 `ModuleIOSim.java` 中，而不是 `TunerConstants.java` 中。

Users who wish to characterize acceleration gains (k_a) or turn gains can choose to use the full [SysId](#) application. The project includes auto routines for each of the four required SysId tests. Two options are available to load data in SysId:

希望测定加速度增益 (k_a) 或转向增益的用户，可以选择使用完整的 [SysId](#) 应用。项目为所需的四项 SysId 测试分别提供了自动程序。有两种方式可将数据加载到 SysId:

- Export the Hoot log file as described [here](#).

按[此处](#)的说明导出 Hoot 日志文件。

- Export the AdvantageKit log file as described [here](#). Note that AdvantageKit values are logged in radians while Phoenix requires rotations to be used. Gains must be converted appropriately.

按[此处](#)的说明导出 AdvantageKit 日志文件。注意，AdvantageKit 的值以弧度记录，而 Phoenix 要求使用转数。必须适当转换增益。

Tip / 提示

The built-in SysId routines can be easily adapted to characterize the turn motor feedforward or the angular motion of the robot (for example, to estimate the robot's [moment of inertia](#)). The code below shows how the `runCharacterization` method can be adapted for these use cases.

可以方便地修改内置 SysId 流程，用于测定转向电机的前馈或机器人的角运动（例如估算机器人的[转动惯量](#)）。下面的代码展示了如何针对这些使用场景修改 `runCharacterization` 方法。

```
java
```

```
/** Characterize turn motor feedforward. */
public void runCharacterization(double output) {
    io.setDriveOpenLoop(0.0);
    io.setTurnOpenLoop(output);
}

/** Characterize robot angular motion. */
public void runCharacterization(double output) {
    io.setDriveOpenLoop(output);
    io.setTurnPosition(new Rotation2d(constants.LocationX, constants.LocationY).plus(Rotation2d.kCCW_Pi_2))
;
}
```

Wheel Radius Characterization / 车轮半径特性测定

The effective wheel radius of a robot tends to change over time as wheels are worn down, swapped, or compress into the carpet. This can have significant impacts on odometry accuracy. We recommend regularly recharacterizing wheel radius to combat these issues.

随着车轮磨损、更换或压入地毯，机器人的有效车轮半径往往会随时间变化。这可能显著影响里程计（odometry）的准确性。我们建议定期重新测定车轮半径，以应对这些问题。

The project includes an automated wheel radius characterization routine, which only requires enough space for the robot to rotate in place.

项目包含一个自动车轮半径测定流程，只需要足够让机器人原地旋转的空间。

1. Place the robot on carpet. Characterizing on a hard floor may produce errors in the measurement, as the robot's effective wheel radius is affected by carpet compression.

将机器人放在地毯上。由于地毯压缩会影响机器人的有效车轮半径，在硬地面上测定可能造成测量误差。

2. Select the "Drive Wheel Radius Characterization" auto routine.

选择 "Drive Wheel Radius Characterization" 自动程序。

3. Enable the robot in autonomous. The robot will slowly rotate in place.

在 autonomous 模式下启用机器人。机器人会缓慢原地旋转。

4. Disable the robot after at least one full rotation.

机器人至少完成一整圈旋转后，将其禁用。

5. Check the console output for the measured wheel radius, and copy the value to `kWheelRadius` in `TunerConstants.java`.

在控制台输出中查看测得的车轮半径，并将该值复制到 `TunerConstants.java` 中的 `kWheelRadius`。

Drive/Turn PID Tuning / 驱动 / 转向 PID 调参

The project includes default gains for the drive velocity PID controllers and turn position PID controllers, which can be found in the `steerGains` and `driveGains` configs in `TunerConstants.java`. These gains should be tuned for each robot.

项目包含驱动速度 PID 控制器和转向位置 PID 控制器的默认增益，可在 `TunerConstants.java` 中的 `steerGains` 和 `driveGains` 配置中找到。这些增益应针对每台机器人进行调整。

Info / 说明

The AdvantageKit template requires different PID gains than CTRE's default swerve code, because it applies the swerve gear ratio using the TalonFX(S) firmware and not on the RIO.

AdvantageKit 模板所需的 PID 增益与 CTRE 默认 swerve 代码不同，因为它通过 TalonFX(S) 固件应用 swerve 减速比，而不是在 RIO 上应用。

Tip / 提示

More information about PID tuning can be found in the [WPILib documentation](#).

有关 PID 调参的更多信息，请参阅 [WPILib 文档](#)。

We recommend using AdvantageScope to plot the measured and setpoint values while tuning. Measured values are published to the `/RealOutputs/SwerveStates/Measured` field and setpoint values are published to the `/RealOutputs/SwerveStates/SetpointsOptimized` field.

我们建议在调参时使用 AdvantageScope 绘制测量值和 setpoint。测量值发布到 `/RealOutputs/SwerveStates/Measured` 字段，setpoint 发布到 `/RealOutputs/SwerveStates/SetpointsOptimized` 字段。

Info / 说明

The PID gains used in simulation can be tuned using the same method. **Simulation gains are stored in ModuleIOSim.java instead of TunerConstants.java.**

仿真中使用的 PID 增益可以用相同方法调整。仿真增益存储在 ModuleIOSim.java 中，而不是 TunerConstants.java 中。

Max Speed Measurement / 最大速度测量

The effective maximum speed of a robot is typically slightly less than the theoretical max speed based on motor free speed and gearing. To ensure that the robot remains controllable at high speeds, we recommend measuring the effective maximum speed of the robot.

机器人的有效最大速度通常略低于根据电机空载转速和传动比计算的理论最大速度。为确保机器人在高速时仍可控，我们建议测量机器人的有效最大速度。

1. Set `kSpeedAt12Volts` in `TunerConstants.java` to the theoretical max speed of the robot based on motor free speed and gearing. This value can typically be found on the product page for your chosen swerve modules.

将 `TunerConstants.java` 中的 `kSpeedAt12Volts` 设置为根据电机空载转速和传动比计算的机器人理论最大速度。通常可以在所选 swerve 模块的产品页面中找到此值。

2. Place the robot in an open space.

将机器人放在开阔空间中。

3. Plot the measured robot speed in AdvantageScope using the `/RealOutputs/SwerveChassisSpeeds/Measured` field.

使用 `/RealOutputs/SwerveChassisSpeeds/Measured` 字段，在 AdvantageScope 中绘制机器人的实测速度。

4. In teleop, drive forward at full speed until the robot's velocity is no longer increasing.

在 teleop 模式下，以全速向前驾驶，直到机器人速度不再增加。

5. Record the maximum velocity achieved and update the value of `kSpeedAt12Volts`.

记录达到的最大速度，并更新 `kSpeedAt12Volts` 的值。

Slip Current Measurement / 打滑电流测量

The value of `kSlipCurrent` can be tuned to avoid slipping the wheels.

可以调整 `kSlipCurrent` 的值，避免车轮打滑。

1. Place the robot against the solid wall.

将机器人紧靠坚固的墙壁放置。

2. Using AdvantageScope, plot the current of a drive motor from the `/Drive/Module.../DriveCurrentAmps` key, and the velocity of the motor from the `/Drive/Module.../DriveVelocityRadPerSec` key.

使用 AdvantageScope，通过 `/Drive/Module.../DriveCurrentAmps` 键绘制一个驱动电机的电流，并通过 `/Drive/Module.../DriveVelocityRadPerSec` 键绘制该电机的速度。

3. Accelerate forward until the drive velocity increases (the wheel slips). Note the current at this time.

向前加速，直到驱动速度增加（车轮打滑）。记录此时的电流。

4. Update the value of `kSlipCurrent` to this value.

将 `kSlipCurrent` 更新为此值。

PathPlanner Configuration / PathPlanner 配置

The project includes a built-in configuration for [PathPlanner](#), located in the constructor of `Drive.java`. You may wish to manually adjust the following values:

项目内置了 [PathPlanner](#) 配置，位于 `Drive.java` 的构造函数中。你可能需要手动调整以下值：

- Robot mass, MOI, and wheel coefficient as configured at the top of `Drive.java`

`Drive.java` 顶部配置的机器人质量、转动惯量（MOI）和车轮系数。

- Drive PID constants as configured in `AutoBuilder`.

`AutoBuilder` 中配置的驱动 PID 常量。

- Turn PID constants as configured in `AutoBuilder`.

`AutoBuilder` 中配置的转向 PID 常量。

Customization / 自定义

Setting Odometry Frequency / 设置 odometry 频率

By default, the project runs at **100Hz** on the RIO CAN bus and **250Hz** on CAN FD buses. These values are stored at the top of `Drive.java` and can be freely customized. The project configures all devices to minimize CAN bus utilization, but we recommend monitoring utilization carefully when increasing frequency.

默认情况下，项目在 RIO CAN 总线上以 **100Hz** 运行，在 CAN FD 总线上以 **250Hz** 运行。这些值存储在 `Drive.java` 顶部，可以自由自定义。项目会配置所有设备以尽量降低 CAN 总线利用率，但我们建议在提高频率时仔细监测总线利用率。

Custom Gyro Implementations / 自定义陀螺仪实现

The project defaults to the Pigeon 2 gyro, but can be integrated with any standard gyro. An example implementation for a NavX is included.

项目默认使用 Pigeon 2 陀螺仪，但可以集成任意标准陀螺仪。项目包含一个 NavX 的示例实现。

To change the gyro implementation, switch `new GyroIOPigeon2()` in the `RobotContainer` constructor to any other implementation. For example, the `GyroIONavX` implementation is pre-configured to use a NavX connected to the MXP SPI port. See the page on [IO interfaces](#) for more details on how hardware abstraction works.

要更改陀螺仪实现，请将 `RobotContainer` 构造函数中的 `new GyroIOPigeon2()` 切换为其他实现。例如，`GyroIONavX` 实现已预先配置为使用连接到 MXP SPI 端口的 NavX。有关硬件抽象如何工作的更多详情，请参阅 [IO 接口](#) 页面。

The `PhoenixOdometryThread` class reads high-frequency gyro data for odometry alongside samples from drive encoders. This class supports both Phoenix signals and generic signals. Note that the gyro should be configured to publish signals at the same frequency as odometry. Call `registerSignal` with a double supplier to create a queue, as shown in the `GyroIONavX` implementation:

`PhoenixOdometryThread` 类会同时读取用于 odometry 的高频陀螺仪数据和驱动编码器样本。此类既支持 Phoenix 信号，也支持通用信号。注意，应将陀螺仪配置为以与 odometry 相同的频率发布信号。使用一个 `double supplier` 调用 `registerSignal` 来创建队列，如 `GyroIONavX` 实现所示：

java

```
Queue<Double> yawPositionQueue = PhoenixOdometryThread.getInstance().registerSignal(navX::getAngle);
```

Info / 说明

Reference the full GyroIONavX implementation for an example of how to create a timestamp queue and update the odometry inputs for the gyro.

有关如何创建时间戳队列并更新陀螺仪的 odometry 输入的示例，请参考完整的 GyroIONavX 实现。

Custom Module Implementations / 自定义模块实现

The template project includes multiple IO implementations for different hardware arrangements, as listed below. The selected IO implementation can be changed in RobotContainer.

模板项目包含多种适用于不同硬件组合的 IO 实现，如下所列。可以在 RobotContainer 中更改选用的 IO 实现。

- ModuleIOTalonFX: TalonFX drive controllers, TalonFX turn controllers, and CANcoders (default)

ModuleIOTalonFX: TalonFX 驱动控制器、TalonFX 转向控制器和 CANcoder（默认）。

- ModuleIOTalonFXS: TalonFXS drive controllers, TalonFXS turn controllers, and CANdi

ModuleIOTalonFXS: TalonFXS 驱动控制器、TalonFXS 转向控制器和 CANdi。

The implementations of ModuleIOTalonFX and ModuleIOTalonFXS can be freely customized to support alternative hardware configurations, such as mixing and matching the TalonFX/TalonFXS, using an alternative encoder, or using a Spark Max/Flex instead of a CTRE controller. **We recommend copying any configuration for alternative devices directly from an existing IO implementation whenever possible** (e.g. copying the CANcoder configuration from ModuleIOTalonFX to ModuleIOTalonFXS). When integrating with Spark devices, please see the example IO implementation in the ModuleIOSpark class of the [Spark Swerve Template](#).

可以自由自定义 ModuleIOTalonFX 和 ModuleIOTalonFXS 的实现，以支持其他硬件配置，例如混用 TalonFX/TalonFXS、使用其他编码器，或用 Spark Max/Flex 替代 CTRE 控制器。我们建议尽可能直接从现有 IO 实现中复制其他设备的配置（例如将 CANcoder 配置从 ModuleIOTalonFX 复制到 ModuleIOTalonFXS）。集成 Spark 设备时，请参阅 [Spark Swerve 模板](#) 中 ModuleIOSpark 类的示例 IO 实现。

As described in the previous section, the PhoenixOdometryThread supports non-Phoenix signals through the registerSignal method. This allows devices from different vendors to be freely mixed.

如上一节所述，PhoenixOdometryThread 通过 registerSignal 方法支持非 Phoenix 信号，因此可以自由混用不同厂商的设备。

By default, the project uses a CANcoder/CANdi in remote/fused/sync mode. When using another absolute encoder (such as a duty cycle encoder or HELIUM Canandmag), we recommend resetting the relative encoder based on the absolute encoder; the relative encoder can then be used for PID control. In this case, the following changes are required:

默认情况下，项目在 remote/fused/sync 模式下使用 CANcoder/CANdi。使用其他绝对编码器（如占空比编码器或 HELIUM Canandmag）时，我们建议根据绝对编码器重置相对编码器；随后可以使用相对编码器进行 PID 控制。在这种情况下，需要进行以下更改：

1. Create the encoder object in ModuleIOTalonFX and configure it appropriately.

在 ModuleIOTalonFX 中创建编码器对象，并进行适当配置。

2. Change the feedback sensor source of the turn controller:

更改转向控制器的反馈传感器来源：

```
java
```

```
turnConfig.Feedback.FeedbackSensorSource = FeedbackSensorSourceValue.RotorSensor;
```

3. Replace RotorToSensorRatio with SensorToMechanismRatio as shown below:

将 RotorToSensorRatio 替换为 SensorToMechanismRatio，如下所示：

java

```
turnConfig.Feedback.SensorToMechanismRatio = constants.SteerMotorGearRatio;
```

4. Reset the relative encoder position at startup:

启动时重置相对编码器位置：

java

```
tryUntilOk(5, () -> turnTalon.setPosition(customEncoder.getPositionRotations(), 0.25));
```

Profiled Turning PID / 带运动曲线的转向 PID

By default, the project uses standard PID controllers for turn control. Users may choose to replace the standard control request with [Motion Magic](#) or [Motion Magic Expo](#) control requests. To implement this, simply replace the position request in `ModuleIOTalonFX` with the new request type, as shown below. The Motion Magic constraints are already configured in the `ModuleIOTalonFX` constructor, but can be adjusted.

默认情况下，项目使用标准 PID 控制器进行转向控制。用户可以选择将标准控制请求替换为 [Motion Magic](#) 或 [Motion Magic Expo](#) 控制请求。要实现这一点，只需将 `ModuleIOTalonFX` 中的位置请求替换为新的请求类型，如下所示。Motion Magic 约束已在 `ModuleIOTalonFX` 构造函数中配置，但可以调整。

java

```
private final MotionMagicVoltage positionVoltageRequest = new MotionMagicVoltage(0.0);
private final MotionMagicTorqueCurrentFOC positionTorqueCurrentRequest = new MotionMagicTorqueCurrentFOC(0.0);
```

Vision Integration / 视觉集成

The Drive subsystem uses WPILib's [SwerveDrivePoseEstimator](#) class for odometry updates. The subsystem exposes the `addVisionMeasurement` method to enable vision systems to publish samples.

Drive subsystem 使用 WPILib 的 [SwerveDrivePoseEstimator](#) 类更新 odometry。该 subsystem 对外提供 `addVisionMeasurement` 方法，使视觉系统能够发布测量样本。

Users migrating from CTRE's swerve library should note that the AdvantageKit template uses **standard FPGA timestamps** for pose estimation rather than CTRE's "current time." This means that pose estimates from Limelight or PhotonVision can be passed *directly* to the pose estimator without needing to call [Utils.fpgaToCurrentTime](#).

从 CTRE swerve 库迁移的用户应注意，AdvantageKit 模板使用标准 **FPGA** 时间戳进行 pose 估计，而不是 CTRE 的 "current time"。这意味着来自 Limelight 或 PhotonVision 的 pose 估计可以_直接_传给 pose 估计器，无需调用 [Utils.fpgaToCurrentTime](#)。

Tip / 提示

This project is compatible with AdvantageKit's [vision template project](#), which provides a starting point for implementing a pose estimation algorithm based on Limelight or PhotonVision.

此项目与 AdvantageKit 的[视觉模板项目](#)兼容，该模板为实现基于 Limelight 或 PhotonVision 的 pose 估计算法提供了起点。

Swerve Setpoint Generator / Swerve Setpoint 生成器

The project already includes basic mechanisms to reduce skidding, such as drive current limits and cosine optimization. Users who prefer more control over module skidding may wish to utilize Team 254's swerve setpoint generator. Documentation for using the version of this algorithm bundled with PathPlanner can be found [here](#). The SwerveSetpointGenerator should be instantiated in the Drive subsystem and used in the runVelocity method, as shown below:

项目已包含减少打滑的基本机制，例如驱动电流限制和余弦优化。希望更细致控制模块打滑的用户，可以使用 Team 254 的 swerve setpoint 生成器。[此处](#)提供了 PathPlanner 内附此算法版本的使用文档。应在 Drive subsystem 中实例化 SwerveSetpointGenerator，并在 runVelocity 方法中使用它，如下所示：

java

```
private final SwerveSetpointGenerator setpointGenerator;
private SwerveSetpoint previousSetpoint;

public Drive(...) {
    // ...

    setpointGenerator = new SwerveSetpointGenerator(...);
    previousSetpoint = new SwerveSetpoint(getChassisSpeeds(), getModuleStates(), DriveFeedforwards.zeros(4));

    // ...
}

public void runVelocity(ChassisSpeeds speeds) {
    previousSetpoint = setpointGenerator.generateSetpoint(previousSetpoint, speeds, 0.02);
    SwerveModuleState[] setpointStates = previousSetpoint.moduleStates();

    // ...
}
```

译注：原文代码中的 SwerveModuleState[] 存在拼写错误；本地模板使用的类型是 SwerveModuleState[]。代码块按原文保留。

Advanced Physics Simulation / 高级物理仿真

The project can be easily adapted to utilize Team 5516's [maple-sim](#) library for simulation, which provides a full rigid-body simulation of the swerve drive and its interactions with the field. Check the documentation for more details on how to install and use the library.

可以方便地修改项目，使用 Team 5516 的 [maple-sim](#) 库进行仿真，该库提供 swerve drive 及其与场地交互的完整刚体仿真。有关该库的安装和使用详情，请查看其文档。

Real-Time Thread Priority / 实时线程优先级

Optionally, the main thread can be configured to use [real-time](#) priority when running the command scheduler by removing the comments [here](#) and [here](#) (**IMPORTANT:** You must uncomment *both* lines). This may improve the consistency of loop cycle timing in some cases, but should be used with caution as it will prevent other threads from running during the user code loop cycle (including internal threads required by NetworkTables, vendors, etc).

可选地，可以通过取消[此处](#)和[此处](#)的注释，将主线程配置为在运行 Command scheduler 时使用[实时](#)优先级（重要：必须取消_两行_的注释）。在某些情况下，这可能改善循环周期定时的一致性，但应谨慎使用，因为它会阻止其他线程在用户代码循环周期内运行（包括 NetworkTables、厂商库等所需的内部线程）。

This customization **should only be used if the loop cycle time is significantly less than 20ms**, which allows other threads to continue running between user code cycles. We always recommend **thoroughly testing this change** to ensure that it does not cause unintended side effects (examples include

NetworkTables lag, CAN timeouts, etc). In general, **this customization is only recommended for advanced users** who understand the potential side-effects.

只有在循环执行时间显著小于 **20ms** 时才应使用此自定义设置，这样其他线程才能在用户代码的循环之间继续运行。我们始终建议充分测试此更改，确保不会造成非预期副作用（例如 NetworkTables 延迟、CAN 超时等）。一般来说，此自定义设置仅推荐给理解其潜在副作用的高级用户。

Vision Template / 视觉模板

The vision template project provides a starting point for creating a high-performance vision or pose estimation system compatible with AdvantageKit's deterministic log replay. It includes support for the following features:

视觉模板项目为创建与 AdvantageKit 确定性日志回放兼容的高性能视觉或 pose 估计系统提供了起点。它支持以下功能：

- Integration with both Limelight and PhotonVision
 - 与 Limelight 和 PhotonVision 集成
- Vision simulation via PhotonLib
 - 通过 PhotonLib 进行视觉仿真
- Options for both simple targeting and full pose estimation
 - 提供简单瞄准和完整 pose 估计两种选项
- High-frequency sampling to ensure that every observation is processed (and never duplicated)
 - 高频采样，确保每个观测都被处理（且不会重复处理）
- Efficient logging of observations via structs
 - 通过 struct 高效记录观测
- Advanced filtering options, including automatic scaling of standard deviations
 - 高级过滤选项，包括自动缩放标准差
- Detailed logging of filters, allowing for easy tuning in replay
 - 详细记录过滤器，便于在回放中调参
- **Deterministic replay with a guarantee of accuracy**
 - 确定性回放，并保证准确性

Info / 说明
The AdvantageKit vision template is open-source and fully customizable :
AdvantageKit 视觉模板是开源且完全可自定义的：
• No black boxes: Users can view and adjust all layers of the vision processing stack.
没有黑箱： 用户可以查看和调整视觉处理栈的所有层。
• Customizable: IO implementations can be adjusted to support any hardware configuration.
可自定义： 可以调整 IO 实现以支持任意硬件配置。
• Replayable: Every aspect of the vision processing and filtering logic can be replayed and logged in simulation using AdvantageKit's deterministic replay features with <i>guaranteed accuracy</i>.
可回放： 视觉处理和过滤逻辑的每个方面，都可以使用 AdvantageKit 的确定性回放功能在仿真中回放并记录，且_保证准确性_。

Warning / 警告

This project is provided as a **starting point** that will work reasonably well across a variety of situations, but **must be customized to fit your specific needs**. It is intended as a **platform** on top of which more optimized systems can be designed. The best pose estimation systems account for a wide variety of factors, including:

此项目作为起点提供，在多种情况下都能较好地工作，但必须根据你的具体需求进行自定义。它旨在作为一个平台，供你在此基础上设计进一步优化的系统。最好的 pose 估计系统会考虑多种因素，包括：

- Game design
比赛设计
- Robot design
机器人设计
- Strategic objectives
战略目标
- Field tolerances
场地公差
- ...and more!
.....以及更多因素！

High-quality pose estimation requires frequent iteration of all aspects of the control stack to address these factors, including **cameras, mounts, coprocessors, calibrations, pipelines, communication, and filtering**. This project provides a starting point for only one small part of a well-optimized vision stack.

高质量的 pose 估计需要频繁迭代控制栈的各个方面，以应对这些因素，包括相机、安装结构、协处理器、标定、**pipeline**、通信和过滤。此项目仅为经过充分优化的视觉栈中的一小部分提供起点。

Configuration / 配置

The project is primarily configured via the `VisionConstants` class, with comments explaining the purpose of each field. The selected vision implementation can be changed in the constructor of `RobotContainer`.

项目主要通过 `VisionConstants` 类配置，注释解释了每个字段的用途。可以在 `RobotContainer` 构造函数中更改选用的视觉实现。

Tip / 提示

In addition to pose estimation, this project include an example of simple targeting with AprilTags. For many games and robots, this is a significantly simpler method of accomplishing game objectives. Check the `getTargetX` method of `Vision` and `configureButtonBindings` method of `RobotContainer` for details.

除 pose 估计外，此项目还包含一个使用 AprilTag 进行简单瞄准的示例。对于许多比赛和机器人，这是一种显著更简单的实现比赛目标的方法。详情请查看 `Vision` 的 `getTargetX` 方法和 `RobotContainer` 的 `configureButtonBindings` 方法。

Logging / 日志记录

The vision subsystem logs a large set of outputs that can be used for debugging and tuning. Each camera logs the following fields:

视觉 subsystem 会记录大量可用于调试和调参的输出。每个相机会记录以下字段：

- **TagPoses:** A list of 3D poses representing the set of visible tags. We recommend visualizing this field using the "Vision Target" object on the 3D field tab in AdvantageScope.

TagPoses: 表示可见 AprilTag 集合的 3D pose 列表。我们建议使用 AdvantageScope 的 3D field 选项卡中的 "Vision Target" 对象可视化此字段。

- **RobotPoses:** A list of 3D poses representing the raw pose estimates from the last cycle. We recommend visualizing this field using the "Ghost" object on the 3D field tab in AdvantageScope.

RobotPoses: 表示上一个循环中原始 pose 估计的 3D pose 列表。我们建议使用 AdvantageScope 的 3D field 选项卡中的 "Ghost" 对象可视化此字段。

- **RobotPosesAccepted:** A subset of the RobotPoses list with the set of estimates that passed all stages of filtering.

RobotPosesAccepted: RobotPoses 列表的一个子集，包含通过所有过滤阶段的估计。

- **RobotPosesRejected:** A subset of the RobotPoses list with the set of estimates that were removed during filtering.

RobotPosesRejected: RobotPoses 列表的一个子集，包含过滤过程中被移除的估计。

The Summary table includes identical fields which include samples from every camera.

Summary 表包含相同的字段，其中汇总了每个相机的样本。

Limelight 4 / Limelight 4

This project is compatible with all variants of Limelight by default (in addition to PhotonVision). **Limelight 4** users who wish to take advantage of the built-in IMU for MegaTag 2 should check the [Limelight documentation](#) for details. Note that the template already publishes the robot orientation every loop cycle.

默认情况下，除 PhotonVision 外，此项目还兼容所有型号的 Limelight。希望利用内置 IMU 进行 MegaTag 2 的 **Limelight 4** 用户，应查看 [Limelight 文档](#) 了解详情。注意，模板已在每个循环周期发布机器人朝向。

Info / 说明

Users can configure the IMU mode by importing [LimelightLib](#) or by publishing an integer to the `imumode_set` key in NetworkTables.

用户可以通过导入 [LimelightLib](#)，或向 NetworkTables 中的 `imumode_set` 键发布一个整数，来配置 IMU 模式。

Real-Time Thread Priority / 实时线程优先级

Optionally, the main thread can be configured to use [real-time](#) priority when running the command scheduler by removing the comments [here](#) and [here](#) (**IMPORTANT:** You must uncomment *both* lines). This may improve the consistency of loop cycle timing in some cases, but should be used with caution as it will prevent other threads from running during the user code loop cycle (including internal threads required by NetworkTables, vendors, etc).

可选地，可以通过取消[此处](#)和[此处](#)的注释，将主线程配置为在运行 Command scheduler 时使用[实时](#)优先级（重要：必须取消_两行_的注释）。在某些情况下，这可能改善循环周期定时的一致性，但应谨慎使用，因为它会阻止其他线程在用户代码循环周期内运行（包括 NetworkTables、厂商库等所需的内部线程）。

This customization **should only be used if the loop cycle time is significantly less than 20ms**, which allows other threads to continue running between user code cycles. We always recommend **thoroughly testing this change** to ensure that it does not cause unintended side effects (examples include NetworkTables lag, CAN timeouts, etc). In general, **this customization is only recommended for advanced users** who understand the potential side-effects.

只有在循环执行时间显著小于 **20ms** 时才应使用此自定义设置，这样其他线程才能在用户代码的循环之间继续运行。我们始终建议充分测试此更改，确保不会造成非预期副作用（例如 NetworkTables 延迟、CAN 超时等）。一般来说，此自定义设置仅推荐给理解其潜在副作用的高级用户。

Skeleton Template / 骨架模板

The AdvantageKit skeleton template includes the basic code required to load AdvantageKit, but no subsystems or control logic. It is intended for teams who wish to design a complete project themselves without using a more complete template.

AdvantageKit 骨架模板包含加载 AdvantageKit 所需的基础代码，但没有 subsystem 或控制逻辑。它适用于希望自行设计完整项目，而不使用功能更完整模板的队伍。

Warning / 警告

The project is configured to save log files when running on a real robot. **A FAT32 formatted USB stick must be connected to one of the roboRIO USB ports to save log files.**

该项目已配置为在真实机器人上运行时保存日志文件。要保存日志文件，必须将一个 **FAT32** 格式的 **U** 盘连接到 roboRIO 的某个 **USB** 端口。

We recommend that most users start with one of the other provided template projects:

我们建议大多数用户从其他提供的模板项目之一开始：

- **2026 KitBot Template**: For robots based on the 2026 FIRST KitBot.

2026 KitBot 模板：适用于基于 2026 FIRST KitBot 的机器人。

- **Differential Drive Template**: For other differential drive (tank) robots.

差速底盘模板：适用于其他差速（tank）底盘机器人。

- **Spark Swerve Template**: For swerve drives primarily using the Spark Max and Spark Flex, including NEO, NEO Vortex, or NEO 550 motors.

Spark Swerve 模板：适用于主要使用 Spark Max 和 Spark Flex 的 swerve drive，包括 NEO、NEO Vortex 或 NEO 550 电机。

- **TalonFX Swerve Template**: For swerve drives primarily using TalonFX-based motors like the Falcon 500, Kraken X60, and Kraken X44.

TalonFX Swerve 模板：适用于主要使用基于 TalonFX 的电机（如 Falcon 500、Kraken X60 和 Kraken X44）的 swerve drive。

- **Vision Template**: Example code for running simple vision targeting and pose estimation.

视觉模板：用于运行简单视觉瞄准和 pose 估计的示例代码。

Replay / 回放

◀◀ How To: Traditional Replay / ◀◀ 操作指南：传统回放

Setup / 设置

The AdvantageKit template projects are preconfigured to support replay by changing the `simMode` option in `Constants.java` to `REPLAY`. More broadly, replay requires the following elements in the logger configuration:

AdvantageKit 模板项目已预先配置为：将 `Constants.java` 中的 `simMode` 选项改为 `REPLAY`，即可支持回放。更一般地说，回放需要在 logger 配置中包含以下要素：

- A log file to use as the source, containing the original inputs and outputs:

一个作为数据源使用的日志文件，包含原始输入和输出：

java

```
// The log path can be read from anything, but this method is provided for convenience
String logPath = LogFileUtil.findReplayLog();

// The following sources are used automatically, with these priorities:
//
// 1. The value of the "AKIT_LOG_PATH" environment variable, if set
// 2. The file currently open in AdvantageScope, if available
// 3. The result of the prompt displayed to the user
```

- A replay source such as `WPILOGReader`:

一个回放源，例如 `WPILOGReader`：

java

```
Logger.setReplaySource(new WPILOGReader(logPath));
```

- A data receiver such as `WPILOGWriter`, which will write a new log file containing the new outputs along with the original inputs and outputs:

一个数据接收器，例如 `WPILOGWriter`，它会写入新的日志文件，其中包含新输出，以及原始输入和输出：

java

```
// The addPathSuffix function generates a new filename by adding the suffix.
// If running replay repeatedly, a numeric index is added to the filename instead.
Logger.addDataReceiver(new WPILOGWriter(LogFileUtil.addPathSuffix(logPath, "_sim")));
```

- Optionally, the robot program can be configured to run faster than real-time. This allows log replay to complete faster than the duration of the original log file and **does not affect the accuracy of log replay**.

可选地，可以将机器人程序配置为以快于实时的速度运行。这样可以在短于原始日志时长的时间内完成日志回放，并且不会影响日志回放的准确性。

java

```
setUseTiming(false);
```

Usage / 用法

To launch log replay, start the robot project in [simulation](#). The generated log file will be opened automatically in AdvantageScope (check the API documentation for `WPILOGWriter` for details on customizing this behavior). Replay outputs are stored in the `ReplayOutputs` table alongside the unmodified inputs and outputs (stored in the `RealOutputs` table).

要启动日志回放，请在[仿真](#)中启动机器人项目。生成的日志文件会自动在 AdvantageScope 中打开（有关自定义此行为的详情，请查看 WPILogWriter 的 API 文档）。回放输出存储在 ReplayOutputs 表中，与未经修改的输入和输出（存储在 RealOutputs 表中）一并保留。

Tip / 提示

The simulation GUI **must be disabled** when running in replay. The GUI is disabled by default in the AdvantageKit template projects.

在回放模式下运行时，必须禁用仿真 GUI。AdvantageKit 模板项目默认禁用该 GUI。

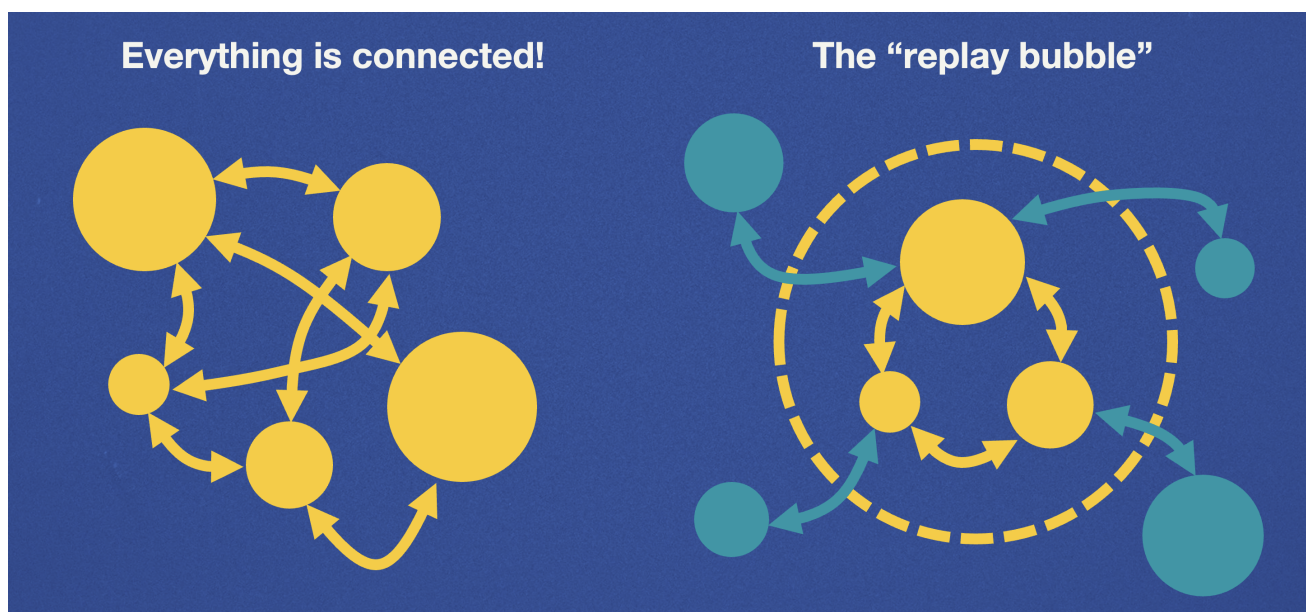
Replay Bubble / 回放边界 (Replay Bubble)

The most straightforward uses of replay involve [logging additional outputs](#). Code can also be modified when running in log replay. However, this use case comes with limitations as **modified outputs cannot affect replayed inputs**. This issue is discussed in more detail in the clip below, which is part of 6328's [2025 Championship Conference](#).

回放最直接的用途包括[记录额外输出](#)。运行日志回放时也可以修改代码。不过，此使用场景存在限制，因为修改后的输出无法影响回放的输入。下面的视频片段更详细地讨论了这个问题，该片段来自 6328 的 [2025 Championship 会议](#)。

[FRC Log Replay and Simulation \(2025\) - FRC 6328 FIRST Championship Conference](#)

[FRC 日志回放与仿真（2025） — FRC 6328 FIRST Championship 会议](#)



Replay bubble / 回放边界

How To: Replay Watch / 操作指南: Replay Watch

Some use cases of log replay benefit from rapid iteration, such as tuning pose estimation algorithms.

Replay watch addresses this use case by automatically updating replayed outputs when the code is modified. An example is shown below, where the replayed output in AdvantageScope updates in real-time as the code is modified:

一些日志回放使用场景受益于快速迭代，例如调试 pose 估计算法。**Replay watch** 通过在代码修改后自动更新回放输出来满足这一需求。下面展示了一个示例，其中随着代码修改，AdvantageScope 中的回放输出会实时更新：

AdvantageKit Replay Watch Demo (Simple)

AdvantageKit Replay Watch 演示（简单示例）

Info / 说明

Check the replay example on [rapid iteration](#) for a more detailed example of this feature in the context of tuning a pose estimation algorithm.

请查看[快速迭代](#)回放示例，其中在调试 pose 估计算法的背景下更详细地展示了此功能。

Usage / 用法

Replay watch requires a custom Gradle task defined in `build.gradle`. This task is included in the AdvantageKit template projects and documented in the installation instructions for [existing projects](#).

Replay watch 需要在 `build.gradle` 中定义一个自定义 Gradle 任务。AdvantageKit 模板项目已包含此任务，[现有项目](#)的安装说明也对其进行了介绍。

1. Configure the project for log replay as normal, such as setting `simMode` in `Constants.java` to `REPLAY` in the AdvantageKit template projects. The `WPILOGWriter` used during replay should be configured using the default, `AUTO`, or `ALWAYS` AdvantageScope open behavior (check the [API docs](#) for details). We also recommend calling `setUseTiming(false)` during setup, as described [here](#).

按通常方式配置项目以进行日志回放，例如在 AdvantageKit 模板项目中将 `Constants.java` 中的 `simMode` 设置为 `REPLAY`。回放期间使用的 `WPILOGWriter` 应配置为默认、`AUTO` 或 `ALWAYS` 的 AdvantageScope 打开行为（详情请参阅 [API 文档](#)）。我们还建议在设置过程中调用 `setUseTiming(false)`，如[此处](#)所述。

2. Open the original log file in AdvantageScope or add the path to the `AKIT_LOG_PATH` environment variable.

在 AdvantageScope 中打开原始日志文件，或将其路径添加到 `AKIT_LOG_PATH` 环境变量中。

3. Run `./gradlew replayWatch` (macOS/Linux) or `gradle.bat replayWatch` (Windows) from the command line.

在命令行中运行 `./gradlew replayWatch` (macOS/Linux) 或 `gradle.bat replayWatch` (Windows)。

4. Log replay will run once, and the resulting log file will open automatically in AdvantageScope.

日志回放将运行一次，生成的日志文件会自动在 AdvantageScope 中打开。

5. When the contents of the `src` directory are modified, log replay will run again automatically and the new log will be opened in AdvantageScope. The time range and visualizations in AdvantageScope will be preserved when loading new data. Note that each iteration of log replay will *overwrite* the previous replayed log file, but will not modify the original log.

当 `src` 目录中的内容被修改时，日志回放会自动再次运行，新日志会在 AdvantageScope 中打开。加载新数据时，会保留 AdvantageScope 中的时间范围和可视化设置。注意，日志回放的每次迭代都会_覆盖_上一次回放生成的日志文件，但不会修改原始日志。

Tip / 提示

Replay watch is limited by the speed at which the robot program can be replayed, which makes it most useful on **short log files** and devices with **fast single-core CPU performance**.

Replay watch 受机器人程序回放速度的限制，因此它最适合较短的日志文件以及 **CPU** 单核性能较强的设备。

Non-Deterministic Data Sources / 非确定性数据源

AdvantageKit replay relies on all data sources being deterministic and synchronized. IO interfaces ensure this is the case for subsystems, and AdvantageKit automatically handles replay for some core WPILib classes (see [here](#) for details). However, it's easy to accidentally use data from sources that are not properly logged. **We recommend regularly testing out log replay during development to confirm that the replay outputs match the real outputs.** Spotting mistakes like this early is the key to fixing them before they become a critical issue at an event.

AdvantageKit 回放依赖所有数据源都具有确定性并保持同步。IO 接口为 subsystem 保证了这一点，AdvantageKit 还会自动处理一些 WPILib 核心类的回放（详情请参阅[此处](#)）。不过，很容易意外使用来自未正确记录的数据源的数据。我们建议在开发期间定期尝试日志回放，确认回放输出与真实输出一致。尽早发现这类错误，是在它们成为赛事中严重问题之前修复它们的关键。

Some common non-deterministic data sources to watch out for include:

需要注意的一些常见非确定性数据源包括：

- Timestamp or Driver Station data used before initializing AdvantageKit. See [here](#).
在初始化 AdvantageKit 之前使用的时间戳或 Driver Station 数据。请参阅[此处](#)。
- Use of raw FPGA timestamps, such as `Timer.getMonotonicTimestamp()`. Use `Timer.getTimestamp()` instead.
使用原始 FPGA 时间戳，例如 `Timer.getMonotonicTimestamp()`。应改用 `Timer.getTimestamp()`。
- NetworkTables data as inputs, including from driver dashboards. See [here](#).
将 NetworkTables 数据用作输入，包括来自驾驶员 dashboard 的数据。请参阅[此处](#)。
- Large hardware libraries like YAGSL or Phoenix 6 swerve, which interact with hardware directly instead of through an IO layer. Try using the AdvantageKit [swerve template project](#) instead.
大型硬件库，例如 YAGSL 或 Phoenix 6 swerve，它们直接与硬件交互，而不经 IO 层。可以尝试改用 AdvantageKit 的 [swerve 模板项目](#)。
- Interactions with the RIO filesystem. Files can be saved and read by the robot code, but incoming data still needs to be treated as an input.
与 RIO 文件系统的交互。机器人代码可以保存和读取文件，但传入的数据仍需要被视为输入。
- Random number generation, which cannot be recreated in a simulator.
随机数生成，无法在仿真器中重现。
- Iteration over unordered collections (such as unordered maps).
遍历无序集合（例如无序映射）。

Multithreading / 多线程

The main robot code logic must be single-threaded to work with log replay. This is because the timing of extra threads cannot be recreated in simulation; threads will not execute at the same rate consistently, especially on different hardware.

机器人代码的主要逻辑必须在单线程中执行，才能使用日志回放。这是因为额外线程的执行时序无法在仿真中重现；线程不会始终以相同的频率执行，尤其是在不同硬件上。

There are two solutions to this issue:

这个问题有两种解决方式：

- Threads are rarely required in FRC code, so start by considering alternatives. Control loop can often run great at 50Hz in the main robot thread, or consider using the closed-loop features of your preferred motor controller instead.

FRC 代码很少需要线程，因此首先应考虑替代方案。控制循环通常可以在机器人主线程中以 50Hz 很好地运行，也可以考虑使用所选电机控制器的闭环功能。

- If a thread is truly required, it must be isolated to an IO implementation. Since the inputs to the rest of the robot code are logged periodically, long-running tasks or high-frequency control loops are possible (but as part of an IO implementation, they cannot be recreated during log replay).

如果确实需要线程，必须将其隔离在一个 IO 实现中。由于机器人其余代码的输入会被周期性记录，因此可以运行耗时较长的任务或高频控制循环（但它们属于 IO 实现的一部分，无法在日志回放期间重现）。

Logging From Threads / 从线程中记录日志

AdvantageKit's logging APIs (i.e. `recordOutput` and `processInputs`) are **not** thread-safe, and should only be called from the main thread. As all values in AdvantageKit are synchronized to the main loop cycle, logging data from other threads would fail to capture values accurately even if this functionality was supported. Instead, threads should only be used in IO implementations (see above) and should record all values as inputs, synchronized appropriately to the main loop cycle in a thread-safe manner.

AdvantageKit 的日志记录 API（即 `recordOutput` 和 `processInputs`）不是线程安全的，应当仅从主线程调用。由于 AdvantageKit 中的所有值都与主循环周期同步，即使支持从其他线程记录数据，这样做也无法准确捕获数值。相反，线程应仅用于 IO 实现（见上文），并将所有值作为输入记录，以线程安全的方式使其与主循环周期适当同步。

Uninitialized Inputs / 未初始化的输入

Timestamps & Driver Station / 时间戳与 Driver Station

Before calling `Logger.start()`, AdvantageKit's built-in input logging is not active. This means that Driver Station data (e.g. from the `DriverStation` or `joystick` classes) and timestamp data (e.g. `Timer.getTimestamp()`) are **not deterministic** and **should not be accessed** by the robot code.

在调用 `Logger.start()` 之前，AdvantageKit 的内置输入日志记录尚未启用。这意味着 Driver Station 数据（例如来自 `DriverStation` 或 `joystick` 类的数据）和时间戳数据（例如 `Timer.getTimestamp()`）不具有确定性，机器人代码不应访问这些数据。

Tip / 提示

Wait to create subsystems or button bindings until after `Logger.start()` is called. For most projects, this can be achieved by instantiating `RobotContainer` at the end of the `Robot` constructor.

应等到调用 `Logger.start()` 之后再创建 subsystem 或按钮绑定。对于大多数项目，可以通过在 `Robot` 构造函数末尾实例化 `RobotContainer` 来实现这一点。

java

```
public class Robot extends LoggedRobot {
    // DANGER: The Intake constructor runs before Logger.start(),
    // so it can access non-deterministic timestamps and DS data.
    private final Intake intake = new Intake();

    // Better: The Flywheel constructor is not called until after
    // Logger.start() has been called in the Robot constructor.
    private final Flywheel flywheel;

    public Robot() {
        // ... (AdvantageKit configuration)
        Logger.start();

        // Since Logger.start() has been called, the Flywheel
        // constructor is free to use timestamps and DS data.
        flywheel = new Flywheel();
    }
}
```

Subsystems / Subsystem

Typically, inputs from subsystems are only updated during calls to `periodic`. Note that this means updated (non-default) input data is not available in the constructor. The solution is to either wait for the first `periodic` call or call `periodic` from within the constructor.

通常，subsystem 的输入仅在调用 `periodic` 时更新。注意，这意味着在构造函数中无法获得更新后的（非默认）输入数据。解决方式是等待第一次 `periodic` 调用，或者在构造函数内部调用 `periodic`。

java

```
public class Example extends SubsystemBase {
    private final ExampleIO io;
    private final ExampleIOInputs inputs = new ExampleIOInputs();

    public Example(ExampleIO io) {
        this.io = io;

        // Inputs are not updated yet
        inputs.position;
    }

    @Override
    public void periodic() {
        io.updateInputs(inputs);
        Logger.processInputs("Example", inputs);

        // Inputs are now updated
        inputs.position;
    }
}
```